COORDENADORIA DE ENGENHARIA DA COMPUTAÇÃO

LUIS FELIPE DOS SANTOS GIANONI
PATRICK NUNES DE SOUZA
MARCOS FAVORETTI JR.

ESTUDO COMPARATIVO DE ALGORITMOS PARA O PROBLEMA DO CAIXEIRO VIAJANTE SOB A ÓTICA DA ANÁLISE COMBINATÓRIA

SOROCABA, SP
2025

LUIS FELIPE DOS SANTOS GIANONI
PATRICK NUNES DE SOUZA
MARCOS FAVORETTI JR.

ESTUDO COMPARATIVO DE ALGORITMOS PARA O PROBLEMA DO CAIXEIRO VIAJANTE SOB A ÓTICA DA ANÁLISE COMBINATÓRIA

Trabalho de conclusão de curso apresentado ao Centro Universitário Facens como exigência parcial para obtenção do diploma de graduação em Engenharia de Computação.
Orientador (a): Marcos Fabio Jardini.

SOROCABA, SP
2025

FICHA CATALOGRÁFICA ELABORADA PELA “BIBLIOTECA FACENS”

Bibliotecária responsável

Eliane da Rocha CRB 8062/8ª

G434e

Gianoni, Luís Felipe dos Santos.

Estudo comparativo de algoritmos para o problema do caixeiro viajante sob a ótica da análise combinatória. / por Luís Felipe dos Santos Gianoni, Marcos Favoretti Jr., Patrick Nunes de Souza. - Sorocaba, SP: [s.n.], 2025.

62f.

Trabalho de Conclusão de Curso (Graduação) – Centro Universitário Facens – Coordenadoria de Engenharia da Computação - Curso de Engenharia da Computação, 2025.

Orientador: Prof. Marcos Fabio Jardini

1. Computação Quântica. 2. Caixeiro Viajante. 3. Análise Combinatória. I. Favoretti Junior, Marcos. II. Souza, Patrick Nunes de Souza. III. Centro Universitário Facens. IV. Título.

CDD 621.39

RESUMO

O trabalho aborda a otimização de rotas com múltiplos destinos, um problema de análise combinatória conhecido como problema do caixeiro viajante (PCV), com o objetivo de analisar e comparar o desempenho de algoritmos de busca clássicos e quânticos. Para a análise, foram implementados o algoritmo exato de força bruta, as metaheurísticas de algoritmo genético e *simulated annealing*, e o algoritmo quântico *iterative quantum approximate optimization algorithm* (IQAOA). A comparação foi realizada por meio de uma aplicação web interativa desenvolvida para este fim, na qual é possível configurar o número de destinos e executar algoritmos para encontrar a rota de menor percurso. O desempenho dos algoritmos foi avaliado segundo as métricas de custo da rota final (distância total), tempo de execução e eficiência na convergência para uma solução. Os resultados indicam que, enquanto o força bruta garante a solução ótima apenas para instâncias com baixo número de destinos devido ao seu custo computacional exponencial, o algoritmo genético e o *simulated annealing* oferecem soluções de alta qualidade em tempo viável para problemas maiores. O algoritmo IQAOA, executado em ambiente simulado, demonstrou capacidade de encontrar soluções competitivas, apontando seu potencial como uma alternativa para problemas de otimização à medida que a tecnologia de computadores quânticos amadurece, contribuindo assim com uma ferramenta prática para a exploração e compreensão destas diferentes estratégias.

Palavras-chave: Computação Quântica, Caixeiro Viajante, Análise Combinatória

ABSTRACT

The study addresses the optimization of routes with multiple destinations, a combinatorial analysis problem known as the traveling salesman problem (TSP), with the objective of analyzing and comparing the performance of classical and quantum search algorithms. For this analysis, the exact brute force algorithm, the metaheuristics genetic algorithm and simulated annealing, and the quantum algorithm iterative quantum approximate optimization algorithm (IQAOA) were implemented. The comparison was conducted through an interactive web application developed for this purpose, in which it is possible to configure the number of destinations and execute algorithms to find the shortest route. The performance of the algorithms was evaluated according to the metrics of final route cost (total distance), execution time, and efficiency in convergence to a solution. The results indicate that while Brute Force guarantees the optimal solution only for instances with a small number of destinations due to its exponential computational cost, the genetic algorithm and simulated annealing offer high-quality solutions within feasible time for larger problems. The IQAOA algorithm, executed in a simulated environment, demonstrated the ability to find competitive solutions, highlighting its potential as an alternative for optimization problems as quantum computing technology matures, thereby contributing a practical tool for the exploration and understanding of these different strategies.

Keywords: Quantum Computing, Traveling Salesman Problem, Combinatorial Analysis

LISTA DE ILUSTRACAO

Figura 1 - Tabuleiro do Icosian game 15
Figura 2 - Gráfico comparativo de escalabilidade entre P e NP 17
Figura 3 - Diagrama de sequência do fluxo dos dados 36
Figura 4 – Traçado de rota no mapa da aplicação 43
Figura 5 – Pontos de interesse selecionados na aplicação 44
Figura 6 – Componente de selecionar algoritmos 44
Figura 7 – Exibição de resultados e métricas 45
Figura 8 – Exemplificando relacionamento de entidades no banco Neo4j 48
Figura 9 - Ferramenta web rota para 5 destinos 57
Figura 10 - Ferramenta web resultado dos algoritmos com 5 destinos 58
Figura 11 - Ferramenta web melhor rota para 10 destinos 59
Figura 12 - Ferramenta web pior rota, IQAOA para 10 destinos 60
Figura 13 - Ferramenta web resultado dos algoritmos com 10 destinos 60
Figura 14 - Ferramenta web melhores rotas para os 20 destinos 62
Figura 15 - Ferramenta web pior rota para os 20 destinos 63
Figura 16 - Ferramenta web resultado dos algoritmos com 20 destinos 63
Figura 17 - Ferramenta web rota do IQAOA em *hardware* clássico com 5 destinos 65
Figura 18 - Ferramenta web rota do IQAOA em *hardware* clássico com 10 destinos 67
Figura 19 - Ferramenta web rota do IQAOA em *hardware* clássico com 20 destinos 68
Figura 20 - Gráfico do tempo de execução dos algoritmos. 70
Figura 21 - Gráfico de distância das rotas dos algoritmos 72
Figura 22 - Gráfico do custo computacional dos algoritmos com 5 destinos 73
Figura 23 - Gráfico do custo computacional dos algoritmos com 10 destinos 75
Figura 24 - Gráfico do custo computacional dos algoritmos com 20 pontos 76
Figura 25 - Gráfico do custo monetário dos algoritmos 78

LISTA DE TABELAS

Tabela 1 - Parâmetros de configuração do algoritmo genético 50
Tabela 2 - Parâmetros de configuração do algoritmo IQAOA 51
Tabela 3 - Parâmetros de configuração do algoritmo *simulated annealing* 52
Tabela 4 - Coordenadas geográficas de 5 destinos 53
Tabela 5 - Coordenadas geográficas de 10 destinos 53
Tabela 6 - Coordenadas geográficas de 20 destinos 54
Tabela 7 - Resultados dos algoritmos com 5 destinos 58
Tabela 8 - Resultados dos algoritmos com 10 destinos 61
Tabela 9 - Resultados dos algoritmos com 20 destinos 64
Tabela 10 - Resultados do IQAOA em *hardware* clássico e quântico com 5 destinos 65
Tabela 11 - Resultados do IQAOA em *hardware* clássico e quântico com 10 destinos 67
Tabela 12 - Resultados do IQAOA em *hardware* clássico e quântico com 20 destinos 68
Tabela 13 - Tempo de execução dos algoritmos.....70
Tabela 14 - Distância das rotas dos algoritmos.....72
Tabela 15 - Custo computacional dos algoritmos com 5 destinos.....74
Tabela 16 - Custo computacional dos algoritmos com 10 destinos.....75
Tabela 17 - Custo computacional dos algoritmos com 20 pontos 76
Tabela 18 - Custo monetário dos algoritmos.....78

LISTA DE SIGLAS

IA	Inteligência Artificial
API	*Application Programming Interface*
APOC	*Awesome Procedures on Cypher*
AWS	*Amazon Web Services*
B&B	*Branch and Bound*
CPU	Processador
DNA	*Deoxyribonucleic Acid*
DP	*Dynamic Programming*
DTO	*Data Transfer Object*
GPU	*Graphics Processing Unit*
IBM	*International Business Machines Corporation*
IQAOA	*Iterative Quantum Approximate Optimization Algorithm*
MIT	*Massachusetts Institute of Technology*
NISQ	*Noisy Intermediate-Scale Quantum*
NP	Tempo Polinomial não Determinístico
P	Tempo Polinomial
QAOA	*Quantum Approximate Optimization Algorithm*
QEC	*Quantum Error Correction*
QPU	*Quantum Processing Unit*
RAM	*Random Access Memory*
REST	*Representational State Transfer*
RSA	*Rivest Shamir Adleman*
SIAM	*Society for Industrial and Applied Mathematics*
SPA	*Single-Page Application*
PCV	Problema do Caixeiro Viajante
UI	*User Interface*
UX	*User Experience*
TSP	*Traveling Salesman Problem*
COBYLA	*Constrained Optimization BY Linear Approximations*
NISQ	*Noisy Intermediate-Scale Quantum*

OX	*ordered crossover*
SDK	*Software Development Kit*
PMX	*partially mapped crossover*
JSON	*JavaScript Object Notation*
IBM	*International Business Machines Corporation*

SUMÁRIO

1 INTRODUÇÃO 12
2 O CAIXEIRO VIAJANTE 14
2.1 CONTEXTUALIZAÇÃO COM GRAFOS 14
2.2 DEFINIÇÃO DO PROBLEMA 15
2.3 COMPLEXIDADE COMPUTACIONAL NP E P 16
2.4 SOLUÇÕES NA ATUALIDADE 17
3 COMPUTAÇÃO QUÂNTICA 19
3.1 FUNDAMENTOS E PRINCÍPIOS DA MECÂNICA QUÂNTICA 19
3.1.1 Superposição 19
3.1.2 Entrelaçamento 20
3.1.3 Interferência 21
3.2 POSSIBILIDADES DE USO E IMPACTOS DA COMPUTAÇÃO QUÂNTICA 22
3.2.1 Descoberta de materiais e farmacologia 22
3.2.2 Criptografia e segurança da informação 23
3.2.3 Inteligência artificial e aprendizado de máquina 23
3.3 DESAFIOS ATUAIS 24
3.3.1 Decoerência e correção de erros quânticos 24
3.3.2 Escalabilidade e desenvolvimento de *hardware* quântico 24
4 MÉTODOS 26
4.1 ALGORITMOS UTILIZADOS 26
4.1.1 Força bruta 26
4.1.2 Algoritmo genético 27
4.1.3 *Simulate anneling* 27
4.1.4 IQAOA 28
4.2 JUSTIFICATIVA DA ESCOLHA DOS ALGORITMOS 29
4.3 AMBIENTE DE EXECUÇÃO 29
4.3.1 Ambiente clássico 29
4.3.2 Ambiente quântico 30
4.3.3 Integração entre ambientes 30

4.4 ARQUITETURA DA APLICAÇÃO 31
4.5 *BACKEND* 31
4.5.1 Pilha tecnológica 31
4.5.2 Design Arquitetural e Contratos de Dados 32
4.5.3 Camadas Arquiteturais 32
4.5.4 Contratos de Dados DTOs 33
4.5.5 Padrões de Projeto e Contratos de Execução 34
4.5.6 Fluxo de dados principal: Cálculo de Rota 35
4.6 DETALHAMENTO DAS ESTRATÉGIAS DOS ALGORITMOS 38
4.6.1 Estratégia utilizada para o algoritmo de força bruta 38
4.6.2 Estratégia utilizada para o algoritmo genético 38
4.6.3 Estratégia utilizada para o algoritmo *simulated annealing* 39
4.6.4 Estratégia utilizada para o algoritmo IQAOA 39
4.7 *FRONTEND* 41
4.7.1 Pilha Tecnológica: Angular e Ecossistema 41
4.7.2 Arquitetura: Componentes, Serviços e Estado Reativo 41
4.7.3 Contratos e Tipos: Sincronia com API e Domínio Interno 42
4.7.4 Componentes Visuais Principais: *Widgets* da Interface 42
4.7.5 Fluxo de Dados da UI: O Caso de Uso de Cálculo de Rota 45
4.8 BANCO DE DADOS 46
4.8.1 Justificativa da Escolha: A Vantagem Nativa dos Grafos 46
4.8.2 Modelo de Dados: Nós e Relacionamentos 47
4.8.3 O Papel Estratégico da Biblioteca APOC: Computação no Banco 48
5 CONFIGURAÇÃO DOS EXPERIMENTOS 50
5.1 PARÂMETROS DOS ALGORITMOS 50
5.2 AMBIENTE DE EXECUÇÃO 52
5.3 CRITÉRIOS DE EXECUÇÃO 52
5.4 PONTOS DE INTERESSE OU DESTINOS 53
6 VALIDAÇÃO E RESULTADOS 56
6.1 APRESENTAÇÃO DOS RESULTADOS NO SISTEMA DE TESTES 56
6.1.1 Comparação entre os algoritmos 57
6.1.1.1 Comparação com 5 pontos 57
6.1.1.2 Comparação com 10 pontos 59
6.1.1.3 Comparação com 20 pontos 61

6.1.2 Comparação entre ambientes de execução 64
6.1.2.1 Comparação por ambiente com 5 pontos.....65
6.1.2.2 Comparação por ambiente com 10 pontos.....66
6.1.2.3 Comparação por ambiente com 20 pontos.....68
6.2 COMPARAÇÃO ENTRE ALGORITMOS E AMBIENTE 69
6.2.1 Tempo de execução 70
6.2.2 Distância das rotas 71
6.2.3 Custo computacional 73
6.2.3.1 Custo computacional para 5 pontos 73
6.2.3.2 Custo computacional para 10 pontos 74
6.2.3.2 Custo computacional para 20 pontos 76
6.2.4 Custo Monetário 77
6.3 DISCUSSÃO GERAL DOS RESULTADOS 79
6.3.1 Entre os algoritmos 79
6.3.2 Entre ambientes de execução 81
7 CONSIDERAÇÕES FINAIS 82
REFERÊNCIAS.....83

1 INTRODUÇÃO

A teoria dos grafos possui grande relevância em diversas áreas do conhecimento, sendo aplicada desde a ciência da computação até a biologia e o estudo de redes sociais. Os grafos representam uma poderosa ferramenta matemática capaz de modelar relações e conexões entre elementos, permitindo a abstração de estruturas complexas por meio de vértices (ou nós) e arestas (ou ligações), que podem ser direcionais ou bidirecionais (Revista Brasileira de Ensino de Física, 2022).

Diversos sistemas e fenômenos do cotidiano podem ser representados por meio de grafos, como redes de computadores, redes neurais, mapas urbanos e redes elétricas. Essa modelagem permite não apenas a visualização das estruturas envolvidas, mas também a aplicação de algoritmos voltados à resolução de problemas complexos (Revista Brasileira de Ensino de Física, 2022). Dentre esses problemas, destacam-se aqueles relacionados à otimização, como o problema do caixeiro viajante (PCV) também conhecido como *traveling salesman problem*, o ciclo hamiltoniano e o ciclo euleriano, que diferem em seus objetivos, restrições e aplicações. Tais problemas serão abordados no capítulo teórico, visando apresentar os fundamentos dessas questões, que serão o problema principal abordado nesse projeto.

Com o avanço da tecnologia, novas abordagens têm sido propostas para lidar com a complexidade de problemas combinatórios. A computação quântica surge como uma dessas alternativas promissoras. Embora ainda se baseie no sistema binário, a computação quântica utiliza fenômenos físicos entre eles, há a superposição e o emaranhamento quântico para operar seus dados por meio de qubits (ou bit quântico). Essas propriedades permitem que o computador quântico processe múltiplos estados simultaneamente, o que pode resultar em uma maior capacidade de processamento em determinados contextos, como na manipulação e análise de grafos complexos (Bernhardt, 2020; Storino, 2021).

Dessa forma, surgem questionamentos importantes, como: qual seria o modelo de processamento mais adequado para cada tipo de problema envolvendo grafos? Em que situações a computação quântica apresenta vantagens significativas em relação aos métodos clássicos? Tais questões serão analisadas ao longo deste

trabalho de forma detalhada, buscando compreender e propor as melhores soluções para problemas específicos por meio da aplicação de algoritmos apropriados.

Assim, a partir do próximo capítulo, são apresentados os fundamentos teóricos que sustentam o desenvolvimento deste trabalho. O Capítulo 2 aborda o PCV, discutindo seus fundamentos, sua relação com a teoria dos grafos e os desafios associados à sua complexidade computacional. Em seguida, o Capítulo 3 explora os princípios da Computação Quântica, destacando os fenômenos de superposição, entrelaçamento e interferência, além de discutir suas aplicações e limitações no contexto da otimização combinatória.

O Capítulo 4 detalha a metodologia adotada, apresentando os algoritmos utilizados, a arquitetura da aplicação web desenvolvida e a configuração dos ambientes de execução. Já o Capítulo 5 descreve os experimentos realizados e os parâmetros definidos para os testes, enquanto o Capítulo 6 apresenta a validação e análise dos resultados obtidos. Por fim, o Capítulo 7 reúne as considerações finais, sintetizando as principais conclusões e apontando possíveis direções para pesquisas futuras.

2 O CAIXEIRO VIAJANTE

Os grafos constituem estruturas matemáticas fundamentais na ciência da computação e em diversas áreas do conhecimento, pois permitem modelar relações complexas entre elementos de maneira abstrata. Essa versatilidade possibilita sua aplicação em redes de transporte, comunicação, logística, biologia, entre outros contextos. Dentro desse universo, destaca-se o PCV, um dos mais estudados na otimização combinatória por sua relevância teórica e prática.

Classificado como *nondeterministic polynomial time hard* (NP), também chamado de tempo polinomial não determinístico difícil, o PCV apresenta elevado grau de complexidade computacional, o que impulsionou o desenvolvimento de algoritmos exatos, heurísticos e metaheurísticos ao longo das décadas. Tais soluções vêm sendo amplamente aplicadas em diferentes domínios, como o planejamento de rotas, o sequenciamento genético e a bioinformática, reforçando a importância desse problema como referência central no estudo de grafos e na análise de eficiência de algoritmos.

2.1 CONTEXTUALIZAÇÃO COM GRAFOS

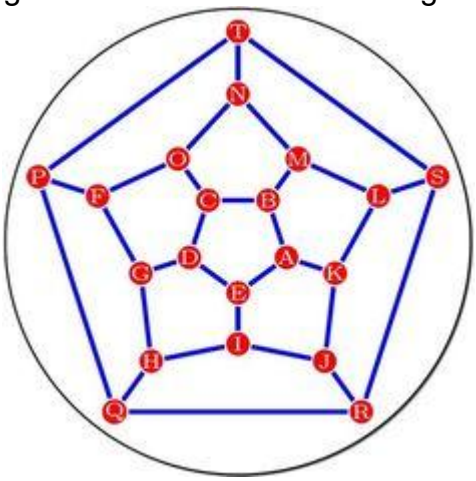
Os grafos são estruturas matemáticas fundamentais nas áreas de matemática discreta e estrutura de dados, amplamente utilizadas na ciência da computação para modelar relações e interações entre elementos (Silva; Oliveira; Costa, 2022). Formalmente, um grafo é composto por um conjunto de vértices (também chamados de nós) e por um conjunto de arestas que conectam pares desses vértices. Essa estrutura permite representar, de maneira abstrata, diversos sistemas do mundo real, como redes de transporte, redes de comunicação, redes sociais, redes elétricas e sistemas logísticos.

Por exemplo, em um sistema de transporte, os vértices podem representar cidades e as arestas, as estradas que as conectam. Na área de estrutura de dados, os grafos são essenciais para o desenvolvimento de algoritmos que resolvem problemas como busca em profundidade e largura, detecção de ciclos, caminhos mínimos e otimização de rotas. É importante observar que diferentes problemas utilizam os grafos de maneiras distintas, destacando sua versatilidade como

ferramenta de modelagem.

Dentro desse contexto, destaca-se o conceito de ciclo hamiltoniano, que é um caminho fechado que passa por todos os vértices do grafo exatamente uma vez, retornando ao ponto de origem. Esse tipo de ciclo foi nomeado em homenagem ao matemático irlandês William Rowan Hamilton, que introduziu a ideia no século XIX (século 19) por meio de um jogo matemático conhecido como *Icosian Game (Jogo Icosiano)*, cujo tabuleiro está representado na Figura 1.

Figura 1 - Tabuleiro do Icosian game



Disponível em: https://fr.pinterest.com/pin/465559680219761838. Acesso em 20 de Abr. 2025.

De acordo com Santos (2016), esse problema teve origem por volta de 1850, e o adjetivo "hamiltoniano" foi atribuído em referência ao trabalho de Hamilton, que modelou o jogo com base em um grafo dodecaédrico. Os vértices foram rotulados como cidades, e o objetivo do jogo era que um viajante visitasse todas as 20 cidades sem repetir nenhuma. A existência de um ciclo hamiltoniano em um grafo está diretamente relacionada a diversos problemas práticos, sendo um dos elementos centrais no PCV.

2.2 DEFINIÇÃO DO PROBLEMA

O PCV é um dos mais estudados na área de otimização combinatória e teoria dos grafos. Sua formulação clássica consiste em encontrar o caminho mais curto que permita a um caixeiro visitar um conjunto de cidades, passando por cada uma exatamente uma vez e retornando ao ponto de partida.

O nome do problema deriva da realidade dos vendedores ambulantes do século XIX, que percorriam diversas localidades para comercializar seus produtos e, por isso, precisavam planejar rotas eficientes que reduzissem o tempo de deslocamento e os custos de viagem.

> Segundo Perkins (2021), as origens do Problema do Caixeiro Viajante, ou TSP, remontam ao início do século XIX, mas foi formalizado matematicamente pela primeira vez por um indivíduo chamado Merrill M. Flood em 1930, que tentava resolver um problema de roteamento de ônibus escolares. Posteriormente, adotado por Hassler Whitney em Princeton sob o apelido de "problema dos 48 estados", no qual a rota mais curta era considerada para visitar todos os 48 Estados Unidos contíguos, recebeu finalmente a expressão atual "TSP" em um relatório da *RAND Corporation* de 1949.

Além do seu valor histórico, o PCV também chamado de *traveling salesman problem* (TSP) é relevante por suas inúmeras aplicações em áreas como genética (sequenciamento de DNA, *deoxyribonucleic acid*, ácido desoxirribonucleico), robótica (planejamento de trajetórias), logística, engenharia, redes e até arte computacional.

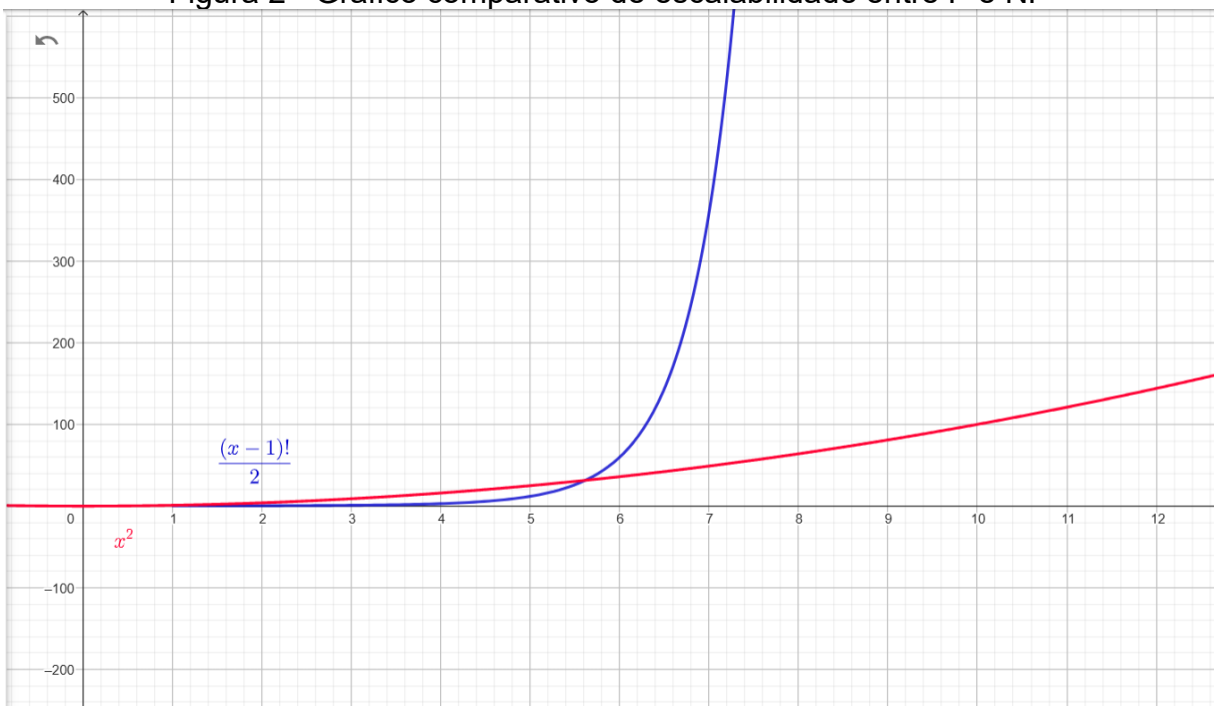
2.3 COMPLEXIDADE COMPUTACIONAL NP E P

O PCV pertence à classe dos chamados problemas NP-hard, sendo considerado NP-completo em sua versão de decisão, que busca determinar se existe um percurso com custo inferior ou igual a um valor previamente estabelecido. A classificação como NP-completo indica que o problema pertence ao conjunto NP, ou seja, ainda que não se conheça um algoritmo que o resolva eficientemente para todas as instâncias, qualquer solução proposta pode ser verificada em tempo polinomial.

De acordo com Perkins (2021), a principal distinção entre os problemas de tempo polinomial (P), como ordenações e buscas simples (com complexidades como O(n), O(n log n), O(n²)), e os problemas de tempo não polinomial, como o PCV, está na escalabilidade da solução como mostra a Figura 2. Problemas NP-hard, como o Caixeiro Viajante, apresentam crescimento exponencial ou fatorial nas possibilidades de resolução. Por exemplo, para *n* cidades, existem $(n-1)!/2$ rotas possíveis distintas. Isso significa que mesmo com um número relativamente pequeno de

cidades, a quantidade de combinações se torna computacionalmente inviável de ser explorada por força bruta.

Figura 2 - Gráfico comparativo de escalabilidade entre P e NP



Fonte: autoria própria.

Essa explosão combinatória impõe uma limitação prática: enquanto algoritmos de tempo polinomial são viáveis mesmo com grandes volumes de dados, algoritmos que demandam tempo exponencial tornam-se rapidamente impraticáveis com o aumento da entrada.

2.4 SOLUÇÕES NA ATUALIDADE

Diante da elevada complexidade do PCV, diversos métodos computacionais têm sido empregados para encontrar soluções viáveis.

Os algoritmos exatos, como o algoritmo de *held-karp* (baseado em programação dinâmica) ou o método *branch and bound* (B&B), garantem a obtenção da melhor solução possível. No entanto, seu tempo de execução cresce exponencialmente com o número de vértices, o que limita sua aplicação a problemas de pequeno ou médio porte.

Como alternativa, surgem os algoritmos heurísticos e metaheurísticos, que não garantem a solução ótima, mas produzem respostas satisfatórias em um tempo

computacional razoável. Segundo Zhou et al. (2019), destacam-se no metaheurísticos os algoritmos genéticos, colônia de formigas, busca tabu e *simulated annealing*. Essas técnicas se baseiam em princípios inspirados na natureza, processos probabilísticos e estratégias adaptativas para explorar o espaço de soluções.

Tais abordagens são amplamente utilizadas em cenários reais, como logística, roteamento de veículos, planejamento de produção, redes de distribuição e bioinformática, onde encontrar uma solução ideal é menos importante do que obter uma resposta suficientemente boa em tempo hábil.

3 COMPUTAÇÃO QUÂNTICA

De acordo com Brassard (1998), a computação quântica é uma tecnologia em constante evolução que explora os princípios da mecânica quântica para processar informações de maneiras fundamentalmente distintas dos computadores clássicos. Enquanto os sistemas computacionais convencionais operam com bits que representam estados definidos (0 ou 1), os qubits, a unidade fundamental de informação quântica, possuem a capacidade de existir em superposição, ou seja, em múltiplos estados simultaneamente.

Essa característica, junto ao fenômeno do entrelaçamento quântico, permite que os computadores quânticos manipulem exponencialmente mais informações e executem cálculos que ultrapassam a capacidade dos sistemas computacionais atuais, esses e mais alguns fundamentos serão mais bem explicados e definidos no decorrer deste capítulo.

Como bem ressaltam Nielsen e Chuang (2010), qualquer tarefa computacional que um computador clássico pode realizar, um computador quântico também pode, mas a recíproca não é verdadeira. De fato, essa tecnologia promissora visa solucionar problemas complexos que para um computador clássico demandariam bilhões de anos de processamento.

3.1 FUNDAMENTOS E PRINCÍPIOS DA MECÂNICA QUÂNTICA

Para compreender a computação quântica, é essencial ter conhecimento dos princípios da mecânica quântica que a sustentam, dentre os quais serão abordados, superposição, entrelaçamento e interferência.

3.1.1 Superposição

De acordo com Grover (2000) a superposição é um dos pilares fundamentais da computação quântica é um conceito que distingue os qubits dos bits clássicos. Em termos simples, a superposição permite que um qubit não se restrinja a um único estado binário (0 ou 1) a qualquer momento, mas sim que exista em uma combinação linear de ambos os estados simultaneamente.

Imagine um bit clássico como um interruptor de luz que está ligado (1) ou desligado (0). Um qubit, por outro lado, pode ser visualizado como um interruptor que está, de alguma forma, ligado e desligado ao mesmo tempo, ou, mais precisamente, em uma proporção de ambos.

Outra analogia para exemplificar o conceito seria imaginar uma moeda girando no ar, enquanto a moeda está girando, ela não é nem "cara" nem "coroa", ela está em um estado de incerteza que abrange ambas as possibilidades. Somente quando a moeda pousa e seu movimento cessa é que seu estado se torna definido. Da mesma forma, um qubit em superposição existe em uma combinação de 0 e 1, com certas probabilidades associadas a cada estado. Conforme explicam Nielsen e Chuang (2010), o estado de um qubit pode ser representado matematicamente como $\alpha|0\rangle + \beta|1\rangle$, onde α e β são amplitudes de probabilidade complexas, e $(|\alpha|^2 + |\beta|^2 = 1)$. Isso significa que o qubit tem uma probabilidade $|\alpha|^2$ de ser medido como 0 e uma probabilidade $|\beta|^2$ de ser medido como 1.

O ato de medição é o que "colapsa" o estado do qubit de sua superposição para um estado clássico definido (0 ou 1). Antes da medição, todas as possibilidades de estado coexistem, mas no momento da observação, apenas uma é revelada probabilisticamente. Essa capacidade intrínseca de existir em múltiplos estados simultaneamente é o que confere aos computadores quânticos um poder inerente de processamento paralelo massivo. Em vez de processar cada estado sequencialmente, um sistema de qubits em superposição pode explorar simultaneamente um número exponencialmente maior de caminhos computacionais. Essa superposição é a base para a aceleração de certos algoritmos, permitindo que eles resolvam problemas em tempo polinomial onde algoritmos clássicos exigiriam tempo exponencial (Rodrigues; Fabricio Filho, 2025).

3.1.2 Entrelaçamento

De acordo com Horodecki (2009) o entrelaçamento quântico é absolutamente crucial para o poder computacional dos sistemas quânticos. Ele descreve uma conexão profunda entre dois ou mais qubits, onde seus estados se tornam ligados de tal forma que o estado de um qubit não pode ser descrito independentemente dos outros, mesmo que estejam fisicamente separados por grandes distâncias.

Como meio de visualização, imagine um par de luvas, tendo uma luva esquerda, sabe instantaneamente que a outra luva é uma luva direita, independentemente de onde ela esteja. No mundo quântico, o entrelaçamento é muito mais profundo. Quando qubits estão entrelaçados, a medição do estado de um deles determina instantaneamente o estado do outro qubit correlacionado, independentemente da distância. Não é uma mera correlação baseada em informação preexistente, mas uma interdependência quântica. Como enfatizam Nielsen e Chuang (2010), o entrelaçamento é um recurso computacional essencial, permitindo que a informação seja distribuída de uma forma que não tem análogo clássico.

Essa correlação instantânea e robusta entre qubits entrelaçados é fundamental para a criação de algoritmos quânticos que exploram a interdependência dos qubits para resolver problemas complexos.

O entrelaçamento é a base para o processamento de informações que transcende as capacidades dos computadores clássicos, possibilitando que as operações em um qubit afetem e revelem informações sobre outros qubits entrelaçados, o que é vital para a eficiência de certas operações quânticas e para a exploração de espaços de solução vastos (Rodrigues; Fabricio Filho, 2025).

3.1.3 Interferência

De acordo com Brassard (1998), a interferência quântica é um princípio fundamental que capacita os algoritmos quânticos a extraírem informações úteis das superposições de estados. Essencialmente, ela permite que as probabilidades de diferentes resultados de um cálculo sejam seletivamente reforçadas ou canceladas. Esse fenômeno é análogo à forma como as ondas (como ondas de luz ou som) interagem, quando duas ondas se encontram, elas podem se somar para criar uma onda de maior amplitude (interferência construtiva) ou se anular mutuamente (interferência destrutiva).

No contexto da computação quântica, os algoritmos são projetados para manipular os qubits de tal maneira que os "caminhos" que levam às soluções corretas para um problema experimentem interferência construtiva, aumentando suas probabilidades de serem medidos. Em contrapartida, os "caminhos" que levariam a soluções incorretas são submetidos à interferência destrutiva, diminuindo drasticamente suas probabilidades. Como descrevem Nielsen e Chuang (2010), a

interferência é o mecanismo que permite aos algoritmos quânticos "navegar" no vasto espaço de superposições de maneira eficiente, amplificando as amplitudes de probabilidade dos estados desejados e minimizando as dos indesejados.

Esse controle preciso sobre as amplitudes de probabilidade é o que confere aos computadores quânticos sua capacidade de resolver certos problemas de forma mais eficiente do que os computadores clássicos. Por exemplo, no algoritmo de Grover para busca em bancos de dados não ordenados, a interferência é usada para amplificar a probabilidade de encontrar o item desejado, reduzindo significativamente o número de passos de busca em comparação com qualquer algoritmo clássico (Grover, 1996). Assim, a interferência quântica atua como um filtro poderoso, destacando as soluções corretas em meio a um oceano de possibilidades. (Brassard 1998).

3.2 POSSIBILIDADES DE USO E IMPACTOS DA COMPUTAÇÃO QUÂNTICA

A computação quântica é um campo em rápida evolução, e desdobrar mais assuntos significa aprofundar em seus fundamentos, aplicações, desafios e no ecossistema que a rodeia. Essa área vem despertando crescente interesse devido ao seu potencial de resolver problemas complexos de forma exponencialmente mais eficiente do que os métodos tradicionais, oferecendo novas perspectivas para setores científicos, tecnológicos e industriais.

A seguir, serão discutidos alguns dos principais campos em que a computação quântica pode ser aplicada, destacando-se a descoberta de materiais e farmacologia, com a simulação precisa de interações moleculares; a criptografia e segurança da informação, com o desenvolvimento de protocolos resistentes à quebra de códigos por computadores quânticos; e a inteligência artificial e aprendizado de máquina, onde algoritmos quânticos podem acelerar o treinamento e aprimorar a capacidade de generalização de modelos complexos.

3.2.1 Descoberta de materiais e farmacologia

No campo da química e ciência dos materiais, a capacidade da computação quântica de simular com precisão o comportamento de moléculas e reações químicas em nível atômico é um avanço notável. Isso se deve à natureza quântica fundamental

das interações moleculares, que computadores clássicos simulam com dificuldade crescente à medida que a complexidade da molécula aumenta. Como apontado por Aspuru-Guzik e Walther (2005), a simulação quântica pode acelerar significativamente a descoberta de novos materiais com propriedades desejadas, como supercondutores e catalisadores, e o desenvolvimento de novos medicamentos, ao permitir um entendimento mais profundo de como as moléculas interagem com alvos biológicos em um nível fundamental.

3.2.2 Criptografia e segurança da informação

A computação quântica tem implicações profundas para a criptografia e a segurança da informação. O advento de algoritmos quânticos como o de *shor* demonstra a capacidade de fatorar números primos grandes de forma eficiente, o que representa uma ameaça direta à segurança dos sistemas de criptografia de chave pública amplamente utilizados atualmente, como o *rivest shamir adleman* (RSA). Essa perspectiva tem impulsionado intensas pesquisas em criptografia pós-quântica, que visa desenvolver e implementar métodos de segurança que sejam resistentes a ataques de computadores quânticos em larga escala (Chen, 2016).

3.2.3 Inteligência artificial e aprendizado de máquina

A convergência da computação quântica com a inteligência artificial (IA) e o *machine learning*, conhecida como aprendizado de máquina quântico, é uma área emergente com grande potencial. Algoritmos quânticos podem, em tese, acelerar o treinamento de modelos de aprendizado de máquina, otimizar redes neurais e processar grandes volumes de dados de maneiras que os computadores clássicos não conseguem. Conforme explorado por Biamonte (2017), essa combinação pode abrir novas fronteiras para a análise de dados, reconhecimento de padrões e até mesmo para o desenvolvimento de IAs mais avançadas, aproveitando a capacidade dos computadores quânticos de explorar espaços de dados complexos de forma mais eficiente.

3.3 DESAFIOS ATUAIS

Apesar de seu imenso potencial transformador, a computação quântica ainda se encontra em seus estágios iniciais de desenvolvimento, confrontando desafios significativos que precisam ser superados para sua plena materialização e aplicação em larga escala. Entre os principais obstáculos destacam-se a decoerência e a necessidade de correção de erros quânticos, que limitam a confiabilidade das operações em qubits, e a escalabilidade e o desenvolvimento de *hardware* quântico, necessários para aumentar o número de qubits e permitir a execução de algoritmos complexos de maneira eficiente.

3.3.1 Decoerência e correção de erros quânticos

Um dos maiores obstáculos enfrentados na construção de computadores quânticos robustos é a decoerência. Esse fenômeno refere-se à perda das propriedades quânticas dos qubits devido à interação indesejada com o ambiente, como flutuações de temperatura ou campos eletromagnéticos (Preskill, 1998). A sensibilidade dos qubits ao ruído ambiental limita drasticamente a vida útil de seus estados quânticos coerentes e, consequentemente, restringe o tempo de execução dos algoritmos quânticos. Para mitigar os efeitos da decoerência e permitir a computação quântica de grande escala, são indispensáveis técnicas de *quantum error correction* (QEC), também chamado de correção de erros quânticos. No entanto, essas técnicas são extraordinariamente complexas e exigem um número substancial de qubits físicos redundantes para codificar um único qubit lógico tolerante a falhas, o que representa um desafio complexo em termos de engenharia e escalabilidade (Gottesman, 1997).

3.3.2 Escalabilidade e desenvolvimento de *hardware* quântico

A construção de computadores quânticos que possuam um grande número de qubits de alta qualidade e com baixas taxas de erro permanece um desafio de engenharia monumental. Atualmente, diversas tecnologias de *hardware* estão sendo ativamente exploradas, incluindo *qubits* supercondutores, íons presos, átomos neutros e qubits topológicos, cada qual apresentando vantagens e desvantagens

distintas em termos de escalabilidade, coerência e conectividade. A fase tecnológica atual é frequentemente referida como a era *noisy intermediate-scale quantum* (NISQ). Essa denominação, cunhada por Preskill (2018), caracteriza os dispositivos quânticos existentes por possuírem um número limitado de *qubits* (tipicamente entre 50 e 100) e serem intrinsecamente suscetíveis a erros, o que impõe restrições significativas sobre a complexidade e a profundidade dos algoritmos que podem ser executados com fidelidade.

4 MÉTODOS

Neste capítulo, serão apresentados os métodos e procedimentos adotados para a construção e avaliação da aplicação Routz, desenvolvida com o propósito de analisar e comparar diferentes algoritmos na resolução do PCV.

Inicialmente, no subcapítulo 4.1, são descritos os algoritmos selecionados para o estudo. Em seguida, o subcapítulo 4.2 aborda os ambientes de execução, tanto clássico quanto quântico.

A arquitetura geral da aplicação é apresentada no subcapítulo 4.3. A partir disso, o capítulo detalha a implementação técnica da solução, começando pelo *backend* em 4.4 e aprofundando as estratégias específicas de implementação de cada algoritmo em 4.5.

Por fim, a construção da interface do *frontend* é detalhada em 4.6, e a modelagem do banco de dados em grafos é justificada e explicada em 4.7.

4.1 ALGORITMOS UTILIZADOS

Esta seção apresenta os principais algoritmos implementados no projeto, detalhando seu funcionamento, características, vantagens e limitações.

4.1.1 Força bruta

O algoritmo de força bruta, ou busca exaustiva, explora todas as soluções possíveis até encontrar aquela que atende aos critérios definidos. Diferente das meta-heurísticas, ele não utiliza atalhos ou aproximações, garantindo que a solução ótima seja encontrada quando existente. No PCV, isso significa testar todas as permutações de caminhos possíveis, calcular o custo de cada um e selecionar o de menor valor.

A principal vantagem é a garantia do resultado ótimo. Contudo, sua complexidade computacional é da ordem de $O(n!)$ (fatorial de n), onde n é o número de cidades. Isso significa que o número de rotas a serem testadas cresce drasticamente a cada cidade adicionada, tornando-o computacionalmente inviável na prática para problemas com um pequeno número de cidades.

No projeto Routz, sua função é servir como base para validar o resultado das soluções em instâncias pequenas.

4.1.2 Algoritmo genético

O algoritmo genético é do tipo meta-heurística inspirada na evolução natural, que busca soluções otimizadas por meio de ciclos iterativos de seleção, cruzamento e mutação. Cada solução candidata (indivíduo) é avaliada por uma função de aptidão (neste caso, o menor custo total da rota). Os indivíduos mais aptos são selecionados para gerar novas soluções (descendentes) na próxima geração.

Para o PCV, uma solução é tipicamente representada como um "cromossomo" que codifica uma permutação de cidades. Operadores de cruzamento (recombinação), como o *ordered crossover* (OX) ou *partially mapped crossover* (PMX), são usados para combinar duas rotas "parentais", enquanto operadores de mutação (como a troca de duas cidades ou a inversão de uma subsequência da rota) introduzem diversidade e evitam a convergência prematura, resultando em uma exploração mais rica do espaço amostral.

O desempenho do algoritmo genético depende da calibração de parâmetros como tamanho da população, taxas de cruzamento e mutação, e critérios de parada. Sua principal vantagem é a robustez, a capacidade de explorar eficientemente espaços de busca complexos e encontrando soluções de alta qualidade para problemas de grande escala onde a força bruta é impraticável.

4.1.3 *Simulate anneling*

O *simulated annealing*, é uma meta-heurística probabilística inspirada no processo de recozimento da metalurgia, onde um material é aquecido e depois resfriado lentamente para atingir um estado de mínima energia.

O algoritmo começa com uma solução inicial aleatória para o PCV. A cada iteração, ele explora uma solução "vizinha" (gerada, por exemplo, pela troca de duas cidades na rota). Se a nova solução for melhor (custo menor), ela é aceita. Se for pior, ela ainda pode ser aceita com uma certa probabilidade, que é função de quão pior ela é e de um parâmetro chamado temperatura.

No início do processo, a temperatura é alta, permitindo que o algoritmo aceite soluções piores com maior frequência. Isso o capacita a sair de mínimos locais e explorar o espaço de soluções de forma ampla. Conforme o algoritmo avança, a temperatura diminui gradualmente (seguindo um "cronograma de resfriamento"),

reduzindo a probabilidade de aceitar movimentos ruins e forçando o algoritmo a convergir para uma solução de baixo custo. A vantagem do *simulate anneling* é sua eficácia em escapar de mínimos locais, mas seu desempenho é altamente sensível à escolha do cronograma de resfriamento e dos parâmetros iniciais.

4.1.4 IQAOA

O *iterative quantum approximate optimization algorithm* (IQAOA) é uma evolução do *quantum approximate optimization algorithm* (QAOA), projetado para otimização combinatória em ambientes quânticos, especialmente em *hardware* quântico de curto prazo (NISQ). Assim como o QAOA, o IQAOA utiliza circuitos quânticos parametrizáveis que exploram superposição e entrelaçamento quântico para navegar pelo espaço de soluções.

A principal diferença é que o IQAOA aplica uma abordagem iterativa que, tipicamente, simplifica o problema a cada passo. Isso pode envolver a eliminação de variáveis ou a fixação de partes da solução com base nos resultados da medição quântica, reaplicando-se então o QAOA ao subproblema restante. Uma vantagem significativa dessa redução é que as iterações subsequentes do problema, sendo menores, podem exigir menos qubits, tornando o IQAOA mais adaptável ao *hardware* com contagens de qubits restritas.

Essa abordagem híbrida combina a execução quântica com um laço de otimização clássico, que ajusta dinamicamente os parâmetros do circuito, especificamente os ângulos β (beta) e γ (gama). Esses ângulos são os "botões de ajuste" do circuito, definindo rotações aplicadas aos qubits. O ângulo γ está associado ao hamiltoniano de custo e controla o "quanto" da estrutura de custo do problema (as distâncias do PCV) é impresso no estado quântico, permitindo ao algoritmo diferenciar soluções boas de ruins. Por sua vez, o ângulo β está associado ao hamiltoniano de mistura e funciona como o motor de exploração que permite ao algoritmo navegar pelo espaço de soluções, "pulando" de uma rota para outra para evitar ficar preso em mínimos locais.

O laço de otimização clássico ajusta dinamicamente esses parâmetros em cada iteração, buscando o balanço ideal entre exploração e refinamento para maximizar a qualidade da solução. Isso aumenta a probabilidade de convergência para soluções de alta qualidade, potencialmente necessitando de circuitos quânticos menos

profundos que o QAOA padrão. A escolha do IQAOA reflete uma busca por maior eficiência e precisão na análise de problemas do PCV em um ambiente controlado, considerando as limitações de *hardware*, incluindo o número de qubits disponíveis.

4.2 JUSTIFICATIVA DA ESCOLHA DOS ALGORITMOS

A seleção dos algoritmos neste estudo buscou contemplar diferentes paradigmas computacionais aplicáveis ao PCV, abrangendo tanto o ambiente clássico quanto o quântico. Essa abordagem permite uma análise comparativa entre métodos tradicionais e emergentes, considerando aspectos de desempenho, escalabilidade e comportamento diante do aumento da complexidade combinatória.

Todavia, o conjunto de algoritmos selecionados contempla diferentes abordagens de resolução do PCV, incluindo força bruta (exato), algoritmo genético (metaheurístico), *simulated annealing* (metaheurístico) e IQAOA (híbrido entre heurístico e meta-heurístico). Essa diversidade de paradigmas computacionais permite explorar distintos métodos de busca desde a exploração completa de todas as possibilidades até técnicas baseadas em aproximação e otimização probabilística possibilitando uma análise mais ampla sobre a eficiência, a qualidade das soluções e o potencial de adaptação de cada algoritmo em diferentes contextos de execução.

4.3 AMBIENTE DE EXECUÇÃO

Para avaliar o desempenho e a aplicabilidade dos algoritmos, a aplicação foi testada em dois ambientes distintos, ambiente (*hardware*) clássico e quântico.

4.3.1 Ambiente clássico

No ambiente clássico, algoritmos como força bruta, *simulate anneling*, IQAOA simulado, e algoritmo genético são executados no computador clássico local. Esse ambiente permite avaliar a escalabilidade dos algoritmos, medir tempo de execução, consumo de memória e comportamento frente a diferentes tamanhos de problemas.

4.3.2 Ambiente quântico

O ambiente quântico do projeto é construído sobre a biblioteca *pennylane* e utiliza o serviço *amazon web services* braket (AWS) para a simulação e execução do algoritmo IQAOA.

A *quantum processing unit* (QPU) escolhida foi a *Ankaa-3*, provisionada pela empresa *Rigetti.* Essa máquina quântica conta com 82 qubits e localizada na California (região us-west-1)

Inicialmente, o projeto utilizava a plataforma *international business machines corporation quantum* (IBM) em conjunto com a biblioteca *Qiskit*. A transição para o ecossistema *pennylane/AWS braket* foi motivada por desafios técnicos de implementação que surgiram após uma migração de plataforma da IBM em 1º de julho de 2025. A adoção da *AWS braket* não apenas resolveu essa questão, mas também proporcionou maior consistência na execução dos testes e facilitou uma melhor integração e mais simplificado com o *backend* da aplicação web.

4.3.3 Integração entre ambientes

A arquitetura da aplicação *Routz* foi projetada para atuar como um orquestrador central, abstraindo a complexidade dos diferentes ambientes de execução para o usuário final.

Para o usuário, essa integração é totalmente transparente. A partir de uma interface *web*, ele define o problema (o conjunto de cidades) e simplesmente seleciona o algoritmo que deseja executar (seja força bruta, algoritmo genético, *simulated annealing* ou IQAOA). O *backend* da plataforma é então responsável por direcionar a requisição ao ambiente apropriado. Os algoritmos clássicos são processados localmente ou em servidores, enquanto o IQAOA é encaminhado para o *AWS braket*.

O ponto chave dessa integração é a padronização dos contratos de saída e entrada de dados. Todos os algoritmos recebem a mesma estrutura da matriz de custos do PCV como *input* (entrada). Independentemente de a execução ter sido clássica ou quântica, sendo responsabilidade da implementação do algoritmo processar esses *inputs* caso os algoritmos exijam mapeamentos específico.

4.4 ARQUITETURA DA APLICAÇÃO

A aplicação *Routz* é estruturada em três componentes principais, sendo eles *backend*, *frontend* e banco de dados. Foi montado 3 containers para executar os processos da aplicação, garantindo portabilidade e padronização do ambiente de execução. A seguir serão abordados cada componente mais detalhadamente.

4.5 *BACKEND*

O componente *backend* da aplicação *Routz*, denominado RoutezAPI, constitui o núcleo computacional e lógico de todo o sistema. Ele foi desenvolvido com a responsabilidade primária de orquestrar os casos de uso, executar os algoritmos de otimização (tanto clássicos quanto quânticos), gerenciar a persistência de dados e prover uma *application programming interface* (API) utilizando o padrão *representational state transfer* (REST), para o consumo de clientes HTTP (protocolo).

Este capítulo detalha as escolhas tecnológicas, o design arquitetural, os padrões de projeto aplicados e, com especial ênfase, os fluxos de dados e os contratos de interface que definem a comunicação interna do sistema.

4.5.1 Pilha tecnológica

A seleção de tecnologias para o RoutezAPI foi orientada pela performance, maturidade do ecossistema e capacidade de integração com ambientes de computação quântica.

- Linguagem e *Framework*: A escolha do *Python* 3.12+ como linguagem base foi natural, dado que é o ecossistema dominante para o desenvolvimento de software quântico. O núcleo da API é construído sobre o *FastAPI* que é um *micro-framework*. A escolha do *FastAPI* se justifica por seu alto desempenho, sistema de injeção de dependências robusto e pela geração automática de documentação interativa *OpenAPI* (swagger), que define o contrato de serviço para o *frontend*.

- Validação de Dados: *Pydantic* é utilizado extensivamente pelo *FastAPI* para definir o *data transfer objects* (DTOs), garantindo a validação de tipos, formatação e regras de negócio em todas as requisições e respostas da API.
- Banco de Dados: Foi empregado o Neo4j, um banco de dados orientado a grafos. Esta escolha é estratégica para o domínio do PCV, pois um banco de dados orientado a grafos permite a modelagem nativa de grafos que é a natureza do problema explorado. Isso torna consultas de caminhos, conectividade e vizinhança muito mais eficientes e intuitivas do que seriam em um modelo relacional tradicional que utiliza de tabelas (linha e coluna).
- Computação quântica: A implementação quântica utiliza a biblioteca *pennylane* como *framework* principal. Sua escolha se deve à capacidade de diferenciação automática de circuitos, uma funcionalidade crucial para o cálculo de gradientes em algoritmos variacionais como o IQAOA, permitindo otimizar os parâmetros do circuito (γ e β). Para a execução em *hardware* quântico real (QPU), o *pennylane* é integrado diretamente com o AWS *braket, software development kit* (SDK). Esta plataforma foi escolhida por fornecer uma interface agnóstica e unificada, permitindo que o mesmo código escrito em *pennylane* seja executado e comparado em QPUs de diferentes modalidades e fornecedores (como IonQ, Rigetti, OQC, etc.).

4.5.2 Design Arquitetural e Contratos de Dados

A arquitetura do *RoutezAPI* é feita com a arquitetura em camadas, com uma separação rigorosa de responsabilidades. A comunicação entre essas camadas é formalizada por contratos de dados explícitos por meio de objetos de transferência de dados (DTOs).

4.5.3 Camadas Arquiteturais

O *backend* ele foi desenvolvido pensando em uma arquitetura em camadas seguindo o padrão de arquitetura limpa. As camadas do sistema serão comentadas abaixo:

1. Camada de Domínio (*Core*): Define as fundações do sistema, incluindo as abstrações (*abstract*) e os DTOs.
2. Camada de Aplicação (*Services*): Contém a lógica de negócio, a orquestração dos casos de uso e as implementações concretas dos algoritmos.
3. Camada de Apresentação (*Delivery*): Expõe a lógica da aplicação via HTTP (rotas do *FastAPI*).
4. Camada de Infraestrutura (Infra): Isola a comunicação com serviços externos, notadamente o banco de dados Neo4j.

4.5.4 Contratos de Dados DTOs

Os objetos de transferência de dados (DTOs), definidos com *pydantic*, são os contratos formais que garantem a validação e a estrutura da informação que flui entre as camadas do sistema e através da API. Estes contratos são divididos em externos (interface da API) e internos (domínio e arquitetura).

Os contratos externos começam pelos AlgoritmosDto, que define a estrutura de entrada da API, contendo a lista de pontos selecionada pelo usuário e o tipo de algoritmo a ser executado. A resposta da API é padronizada pelos algoritmos ResponseDto, que retorna a rota (uma lista de pontos ordenada), o custo total (distância) e uma lista de métricas com seus respectivos resultados.

Para os contratos internos, a entidade de domínio principal é o PontoDTO, que representa um ponto de Interesse com *id*, *name*, latitude e longitude. O MetricaDto é um DTO interno fundamental para o padrão *observer*, definindo a estrutura de um resultado de métrica (com nome, valor, tipo e descrição) que será, por fim, exposto na resposta da API.

Um dos DTOs internos mais importantes para a arquitetura é o MenorCaminhoResultado. Ele serve como o contrato de saída do padrão *strategy* e é crucial para o desacoplamento. A lógica de otimização opera sobre índices e retorna este DTO contendo uma rota e um custo. A camada de serviço, então, utiliza esses índices para reordenar a lista original de PontoDto antes de enviá-la ao cliente.

4.5.5 Padrões de Projeto e Contratos de Execução

Para garantir alta coesão, baixo acoplamento e extensibilidade, a arquitetura de execução dos algoritmos foi construída sobre uma combinação de quatro padrões de projeto clássicos: *Template Method, Strategy, Factory e Observer*. O núcleo dessa arquitetura são as classes base abstratas AlgoritmoBase e MetricasBase.

A classe *AlgoritmoBase* utiliza o padrão *Template Method* para definir o esqueleto principal da execução de um algoritmo. O método público executar atua como o template, ditando a sequência de passos imutável, sendo esses passos a notificação dos observadores das métricas sobre o início da execução, execução da lógica de cálculo específica do algoritmo, notificação dos observadores sobre a finalização e, por fim, o retorno dos resultados processados das métricas.

O *strategy* é aplicado para permitir que este *template* execute diferentes lógicas de cálculo. A "estratégia" é definida pelo método abstrato executar_logica_algoritmo, que as classes filhas são forçadas a implementar. Destrinchando esse contrato, o método recebe como parâmetros uma matriz das distâncias entre os pontos (a dist_matrix) e os próprios pontos.

O método implementado retorna uma tupla que encapsula a sequência de vértices da rota otimizada e o custo total (distância). Esta arquitetura adota um Padrão de Projeto *Template Method*, no qual o *template* é responsável pela gestão do ciclo de vida da execução e pela instrumentação (coleta de métricas). Em contrapartida, as estratégias concretas (e.g., AlgoritmoGeneticoService, ForcaBrutaService) mantêm seu foco exclusivo na lógica de otimização.

Para promover o desacoplamento entre a execução dos algoritmos e a coleta de métricas (como tempo de processamento, consumo de memória, custo quântico, etc.), foi aplicado o Padrão de Projeto *Observer*. Neste padrão, a classe AlgoritmoBase atua como o Sujeito (ou Publicador), emitindo eventos referentes ao seu ciclo de vida. Esta notificação é realizada através dos métodos notificar_inicio_execucao e notificar_fim_execucao, permitindo que Observadores (módulos de métricas) reajam de forma autônoma às mudanças de estado do algoritmo. As classes de métrica podem se inscrever para ouvir esses eventos usando o método adicionar_metrica fornecido pala classe AlgoritmoBase.

A interface do *observer* é definida pela classe MetricasBase. Qualquer classe que implemente MetricasBase deve definir os métodos de contrato

on_inicio_execucao, on_fim_execucao e resultado_final. Este padrão permite que novas métricas como tempo de execução, memória ou preço sejam criadas e "anexadas" a qualquer algoritmo dinamicamente, sem que o algoritmo precise saber quais ou quantas métricas estão o observando. Isso garante uma arquitetura limpa e extensível para monitoramento de desempenho.

Para gerenciar a instanciação das estratégias de algoritmo, foi utilizado o *Factory*, implementado na classe FabricaAlgoritmosService. Esta fábrica expõe um único método de contrato chamado get_algoritmo.

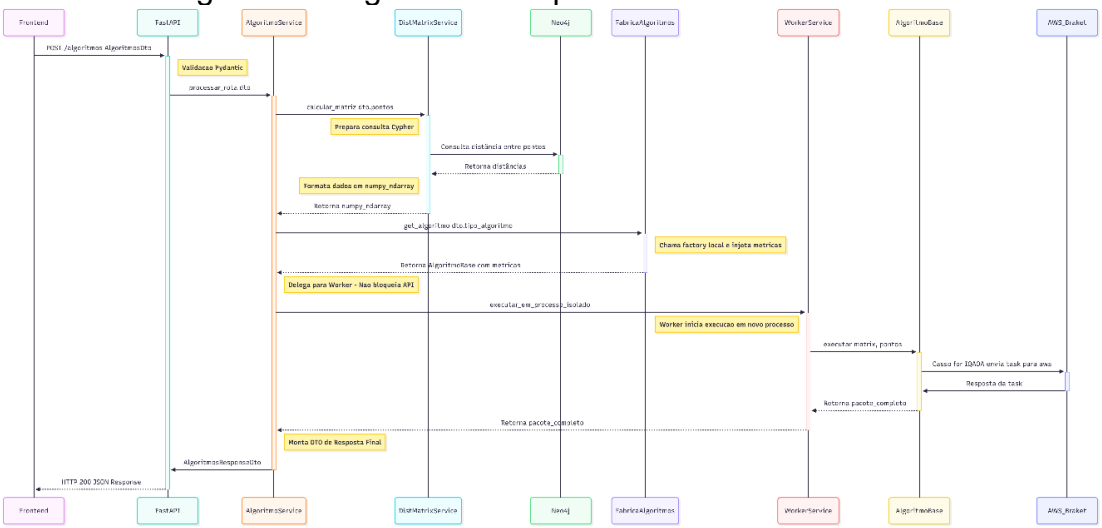
A implementação desta fábrica adota uma decisão de *design* deliberada e fundamental para garantir o isolamento da execução. Uma nova instância do algoritmo solicitada é criada a cada chamada. Este padrão de instanciação sob demanda garante que cada execução de algoritmo seja um processo isolado.

Esse isolamento é crucial para a integridade da coleta de métricas (definida pelo padrão *observer*) em um ambiente web concorrente. Como cada AlgoritmoBase é um novo objeto, seu estado interno e o de suas métricas anexadas (como *timers* ou contadores de memória) não são compartilhados. Isso previne o vazamento ou a contaminação de dados entre requisições paralelas, garantindo que os dados de desempenho sejam de tempo, memória ou custo de QPU sejam exclusivos e íntegros para aquela execução específica.

4.5.6 Fluxo de dados principal: Cálculo de Rota

O fluxo de dados para o principal caso de uso do sistema, o cálculo de rota, é um processo complexo onde passa por 6 ou mais etapas, dependendo de qual algoritmo for selecionado. Este processo é o ponto onde todos os padrões de arquitetura, contratos e serviços convergem para entregar valor, transformando a requisição bruta do usuário em uma solução otimizada, garantindo isolamento, monitoramento e extensibilidade. O diagrama de sequência apresentado na Figura 3 ilustra visualmente esta interação complexa entre os componentes, que é detalhada textualmente a seguir.

Figura 3 - Diagrama de sequência do fluxo dos dados



Fonte: autoria própria

Tudo começa na requisição e validação, a primeira fase na camada de *controller*. O *endpoint* post algoritmos recebe a requisição HTTP contendo o corpo *javascript object notation (JSON)*. Imediatamente, o *FastAPI*, auxiliado pelo *Pydantic*, atua como um rigoroso porteiro, validando e desserializando este JSON para o contrato de entrada AlgoritmosDto. Se os dados forem válidos, o *controller* invoca a camada de serviço, passando o DTO tipado e pronto para o processamento.

Em seguida, entramos na orquestração e transformação de dados, o domínio do AlgoritmoService. Atuando como o grande orquestrador da aplicação, sua primeira tarefa é uma transformação de dados, ele extrai a lista de pontos da requisição e invoca o DistanceMatrixService. Este serviço especializado não calcula as distâncias manualmente, em vez disso, ele se conecta ao banco de dados Neo4j e, por meio de consultas otimizadas utilizando procedimentos da biblioteca *awesome procedures on cypher* (APOC), obtém a menor distância de caminho de cada permutação usando como referência de peso o tamanho da aresta. O Neo4j retorna esses dados, que o serviço então formata na matriz de adjacência necessária para os algoritmos. Em posse dessa matriz de custo, o orquestrador consulta a FabricaAlgoritmosService com o tipo de algoritmo solicitado. Neste ponto, um elo crucial entre os padrões é formado, a fábrica principal atua como uma fachada, delegando a instanciação para uma função *factory* de cada algoritmo. Esta função é responsável não apenas por instanciar a estratégia, mas também por injetar seu conjunto específico de métricas, retornando um objeto AlgoritmoBase novo, isolado e já instrumentado.

Uma das arquiteturas mais críticas para a robustez da aplicação, é a delegação para o *worker* para isolamento de processo de execução dos algoritmos. O AlgoritmoService, atuando como orquestrador, não executa o cálculo diretamente, o que bloquearia o *event loop* da API. Em vez disso, ele delega a tarefa a um *WorkerService*. Passando a instância do algoritmo recém-criada pela fábrica, juntamente com a dist_matrix e a lista de pontos, para este *worker*. O *WorkerService* gerencia um *pool* de processos (*ProcessPoolExecutor*), garantindo que cada cálculo de rota seja executado em um processo completamente isolado no nível do sistema operacional. Esta abordagem permite que múltiplas execuções concorrentes ocorram simultaneamente sem que uma impacte a outra, garantindo a integridade total das métricas, pois cada processo possui seus próprios contadores de tempo e memória.

É dentro deste processo isolado do *worker* que a execução acontece. O *worker* invoca o único método público do contrato, algoritmo_executar. Este é o *Template Method*, que assume o controle, notifica os observadores (notificar_inicio_execucao) e, crucialmente, invoca o método estratégico abstrato executar_logica_algoritmo. Este é o ponto de entrada para a lógica de negócio de cada algoritmo, um processo tão central e detalhado que cada estratégia será destrinchada individualmente no próximo capítulo.

Na quinta fase, a coleta de resultados e retorno ao orquestrador, a lógica da estratégia finaliza seu trabalho e retorna a tupla (rota_ordenada_dtos, custo_final) para o *template method* executar. O método executar (ainda dentro do *worker*) recebe este resultado, notifica os observadores do fim da execução (notificar_fim_execucao) e invoca resultadoFinal de cada métrica anexada. Este pacote completo (DTOs ordenados, custo e a lista de MetricaDto) é retornado pelo *WorkerService* à *thread* principal do AlgoritmoService.

Por fim, na última etapa, ocorre o processo de resposta e serialização. O AlgoritmoService, responsável pela orquestração, recebe os dados previamente processados, ordenados e instrumentados pelo *worker*, encapsulando-os no contrato de saída AlgoritmosResponseDto. Em seguida, a camada de Delivery (*FastAPI*) assume o controle, realizando a serialização desse DTO em uma resposta JSON e enviando-a ao *frontend* com status HTTP 200, concluindo assim o ciclo completo de processamento e entrega da informação.

4.6 DETALHAMENTO DAS ESTRATÉGIAS DOS ALGORITMOS

O capítulo anterior detalhou a arquitetura e o fluxo de dados que permite a execução isolada e polimórfica de qualquer algoritmo. Este capítulo foca no "coração" desse projeto, a implementação interna de cada estratégia, detalhando as particularidades de fluxo de cada método de otimização orginalmente implementado.

4.6.1 Estratégia utilizada para o algoritmo de força bruta

A estratégia ForcaBrutaService, fornece o *benchmark* de precisão exata para o PVC. Seu fluxo interno executar_logica_algoritmo adota uma abordagem específica para o tratamento do ponto de partida, ele fixa a cidade de origem (índice 0) e utiliza a biblioteca *itertools*.*permutations* para gerar permutações apenas para os N-1 pontos restantes. Para cada permutação, um caminho completo é montado (iniciando e terminando em 0), e seu custo total é calculado iterativamente, somando os valores da matriz de adjacência. Este custo é então comparado com o menor custo já registrado na variável min_distance, e a melhor rota (uma lista de índices) é armazenada. Finalmente, antes de retornar, o algoritmo mapeia esta lista de índices com base na lista original de pontos.

4.6.2 Estratégia utilizada para o algoritmo genético

A estratégia do algoritmo genético, começa com a fase de inicialização, onde uma população de 100 indivíduos é criada. A abordagem difere fundamentalmente do Força Bruta, o algoritmo não fixa um ponto de partida. Cada indivíduo é uma permutação aleatória de todos os N índices (de 0 a N-1). O fechamento do ciclo é garantido apenas na fase de avaliação, pela função *calculate*_fitness, que calcula o custo total da rota e soma explicitamente a distância do último gene de volta ao primeiro.

Após a inicialização, o fluxo entra no laço evolutivo principal, que itera por 500 gerações. Em cada geração, o algoritmo executa os operadores genéticos. Primeiramente, o elitismo é aplicado, copiando o melhor indivíduo diretamente para a nova população. Em seguida, a seleção por torneio é usada para escolher os pais,

que então passam pela reprodução via crossover e mutação para gerar os filhos que preencherão o restante da nova população.

Após o término das gerações, o melhor indivíduo encontrado é retornado como resposta da função.

4.6.3 Estratégia utilizada para o algoritmo *simulated annealing*

A estratégia do algoritmo *simulated annealing* implementa uma metaheurística baseada em trajetória. O fluxo de executar_logica_algoritmo adota uma abordagem híbrida para o ponto de partida. Primeiro, uma permutação aleatória de todas as N cidades é criada, em seguida, o índice 0 é localizado e forçado para a posição inicial através de uma troca. Crucialmente, as operações subsequentes de geração de vizinhança (troca de duas cidades) são projetadas para excluir o índice 0, selecionando aleatoriamente apenas índices no intervalo. O fluxo principal é um laço de resfriamento (controlado por taxa_resfriamento=0.995) que contém um laço de iteração interno. A decisão de aceite é o núcleo da lógica. A nova solução é aceita se seu custo for menor ou se uma probabilidade baseada na temperatura ($e^{(-\Delta C/T)}$) for satisfeita

4.6.4 Estratégia utilizada para o algoritmo IQAOA

A estratégia AlgoritmoIQAOAAmazon representa a incursão na computação quântica, implementando uma variante eficiente do QAOA, conhecida como IQAOA (*Indirect* QAOA), em um fluxo híbrido clássico-quântico. O método executar_logica_algoritmo orquestra meticulosamente um processo de quatro passos principais.

O primeiro passo é a preparação do problema quântico. A *dist_matrix*, variável que contém a matriz de adjacência dos pontos, clássica é convertida em um hamiltoniano de custo. Essencialmente, este Hamiltoniano é um operador quântico que "traduz" o problema de otimização para a linguagem da física. No exemplo em contexto cada rota possível é representada como um estado quântico, e o custo dessa rota (a distância total) é codificado como o nível de energia desse estado.

Crucialmente, esta implementação utiliza a codificação indireta (IQAOA), que mapeia o problema para apenas n qubits (um por cidade), uma melhoria significativa

em relação aos n² qubits da codificação QAOA padrão para PCV. O Hamiltoniano construído é associando as distâncias a interações entre pares de qubits, utilizando operadores PauliZ(i) @ PauliZ(j). Um Hamiltoniano de Mistura (*mixer*_h) padrão, baseado em operadores PauliX, também é criado.

O segundo passo é a otimização local dos parâmetros. Para encontrar os ângulos ótimos β (beta) e γ (gama) do QAOA, a implementação utiliza um otimizador clássico COBYLA. A função de custo que este otimizador minimiza (*create_classical_cost_function*) é uma implementação híbrida inteligente, ela executa o circuito quântico parametrizado exclusivamente em um simulador local de alta performance e retorna as probabilidades de todos os estados da base computacional. Em seguida, um loop clássico itera sobre essas probabilidades, cada *bitstring* é decodificado para uma permutação de rota (*decode_permutation*), sua distância clássica real é calculada, e o custo esperado total é retornado ao otimizador. Esta abordagem otimiza diretamente a métrica de negócio.

O terceiro passo é a amostragem final. Após encontrar os parâmetros ótimos localmente, o circuito é executado uma última vez para obter a distribuição de resultados. A implementação oferece flexibilidade através da flag de classe USE_QPU. Se USE_QPU for *false*, o método *get_sampling_device* seleciona novamente o simulador. Isso permite uma execução totalmente local, útil para desenvolvimento, testes e cenários onde o acesso à QPU não é necessário ou desejado. Se USE_QPU for *true*, o mesmo método seleciona o *hardware* quântico real, submetendo uma *task* à *AWS Braket* com os parâmetros já otimizados. Independentemente do dispositivo escolhido (simulador ou QPU), a execução final utiliza *qml.counts* para obter um dicionário de *bitstrings* medidos e suas respectivas contagens, com um total de TOTAL_SHOTS_FINAL disparos. Esta abordagem baseada em contagens é padrão para algoritmos QAOA onde a solução é codificada no estado da base computacional mais provável.

O último passo é o processamento de resultados. O fluxo recebe o dicionário das contagens das amostragens, ordena-o por contagem decrescente e itera sobre os resultados para encontrar o *bitstring* mais provável que corresponda a uma rota válida. A decodificação utiliza uma heurística específica para a codificação indireta, as cidades cujo qubit associado mediu '1' são colocadas primeiro na rota, seguidas pelas cidades que mediram '0'. Finalmente, o método _normalizar_rota é aplicado para

rotacionar o ciclo, garantindo que a rota final seja sempre apresentada começando pelo índice 0.

4.7 *FRONTEND*

A interface do usuário (*Frontend*), denominada *Routez*, constitui a camada de apresentação da aplicação, proporcionando a interação visual e funcional para o usuário final. É responsável por exibir o mapa, permitir a seleção de Pontos de Interesse (POIs), oferecer a escolha do algoritmo de otimização e apresentar os resultados, incluindo a rota otimizada e as métricas de desempenho. Este capítulo detalha a arquitetura, as tecnologias e os principais fluxos de dados que compõem esta interface web moderna e reativa.

4.7.1 Pilha Tecnológica: Angular e Ecossistema

A seleção das tecnologias para o *Routez* foi orientada pela robustez do ecossistema, performance no navegador, ferramentas de desenvolvimento produtivo e capacidade de integração com a API REST do *Backend* (RoutezAPI). A base da aplicação é o framework Angular (v17+), escolhido por sua arquitetura baseada em componentes madura, sistema de injeção de dependência e suporte a *TypeScript*, que garante a segurança de tipos e melhora a manutenibilidade do código.

Para a construção da interface visual, a biblioteca de componentes *PrimeNG* foi adotada, fornecendo um conjunto rico e customizável de que aceleraram o desenvolvimento. A interação com o mapa geográfico é gerenciada pela biblioteca Leaflet, integrada ao Angular via manipulação direta no componente de mapa.

A comunicação com a *RoutezAPI* é padronizada e automatizada pela ferramenta Kubb, que gera automaticamente os modelos *TypeScript* (DTOs) e as funções *client* a partir da especificação *OpenAPI* do *Backend*, garantindo a sincronia dos contratos. Todo o ecossistema é gerenciado pelo Node.js e npm.

4.7.2 Arquitetura: Componentes, Serviços e Estado Reativo

A arquitetura do *Routez* segue as melhores práticas do Angular, promovendo a modularidade e a reatividade. A interface é decomposta em Componentes hierárquicos, onde AppComponent orquestra a página principal AppPage, que contém

os *widgets* visuais. A lógica de negócio, acesso à API e gerenciamento de estado são encapsulados em serviços, desacoplando-os da apresentação. O Gerenciamento de Estado adota uma abordagem moderna e reativa, utilizando *Angular Signals* expostos por serviços dedicados (como PoiStoreService e AlgoritmoStoreService).

Os componentes reagem às mudanças desses *Signals*, atualizando a interface automaticamente. Ações do usuário invocam métodos nos serviços, que modificam os *Signals*, propagando o novo estado pela aplicação de forma eficiente. O *Angular Router* está configurado, permitindo futuras expansões.

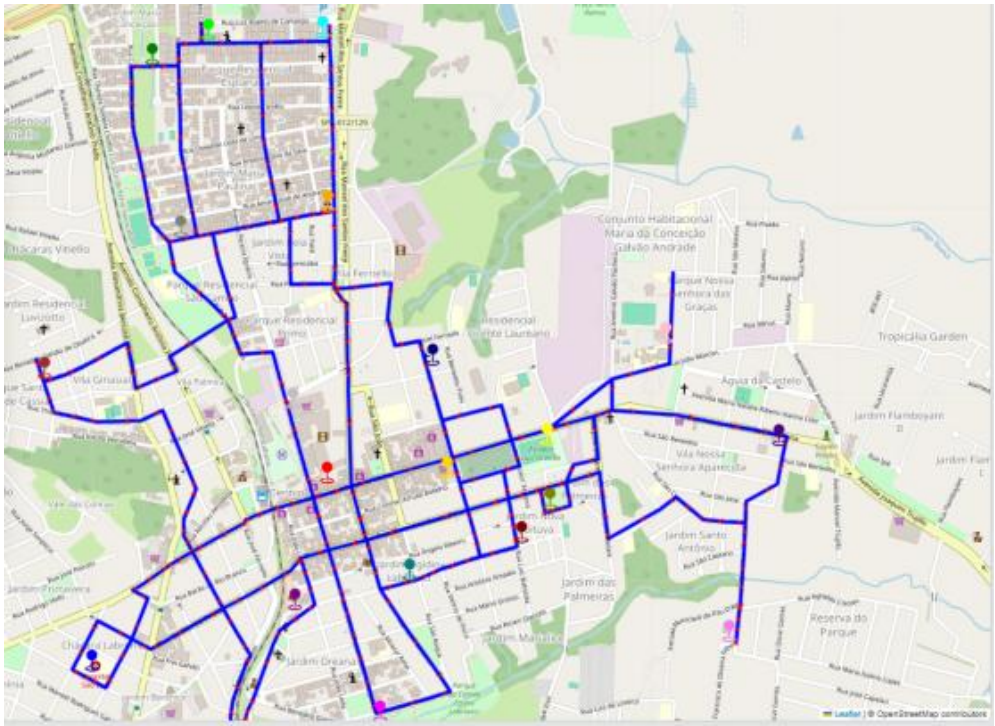
4.7.3 Contratos e Tipos: Sincronia com API e Domínio Interno

A integridade dos dados trocados com o *Backend* e utilizados internamente é garantida por modelos *TypeScript*. Os Modelos de API (DTOs como PontoDTO, AlgoritmosDto, AlgoritmosResponseDto) são gerados automaticamente pela ferramenta Kubb, assegurando a conformidade com o contrato *OpenAPI* exposto por *swagger* da *RoutezAPI*.

4.7.4 Componentes Visuais Principais: *Widgets* da Interface

A arquitetura da interface do usuário (*frontend*) do sistema *Routez* é implementada como uma Aplicação de Página Única (SPA), cuja funcionalidade é orquestrada pela composição modular de *widgets* especializados, sendo a *AppPage* o contêiner central. O elemento geográfico fundamental é o *MapComponent*, que atua como o motor de visualização e interação central. Este componente é responsável por encapsular a renderização do mapa via *Leaflet*, a exibição dos marcadores de Pontos de Interesse (POIs), a captura de eventos de clique do usuário para seleção de coordenadas, e o desenho final da rota otimizada. Sua função na amostragem e visualização dos dados otimizados é ilustrada na Figura 4.

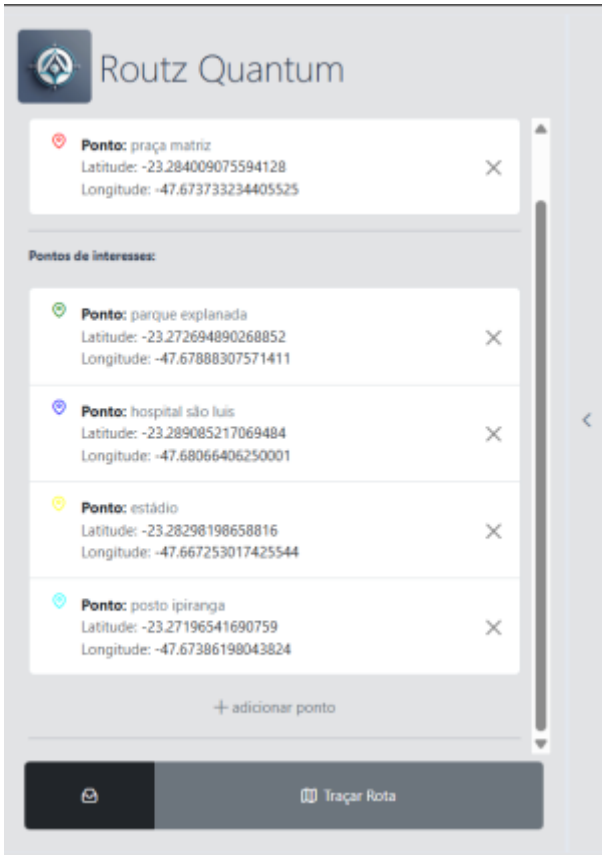
Figura 4 – Traçado de rota no mapa da aplicação



Fonte: autoria própria

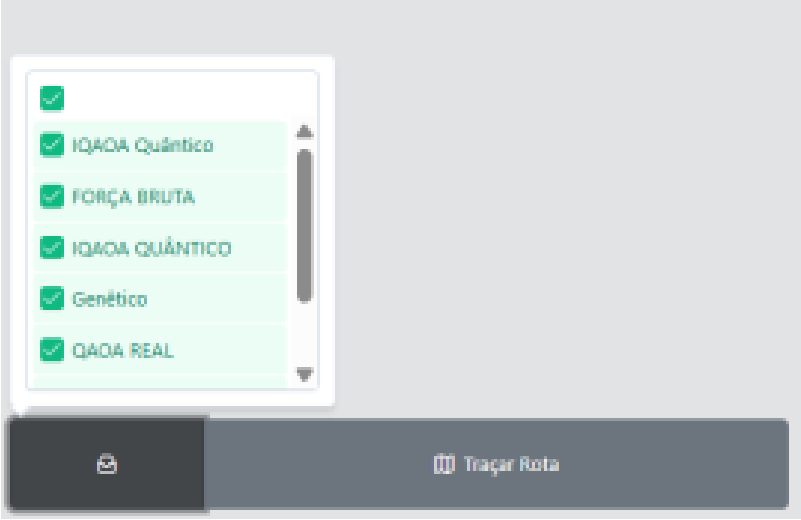
Complementarmente ao mapa, o SideBarComponent estabelece o painel de controle da aplicação. Ele agrega subsistemas essenciais para a configuração da otimização. Primeiramente, o ListaDePontosComponent provê a visualização e gestão interativa dos POIs selecionados pelo usuário como mostra a Figura 5. Em seguida, o SelecionadorDeAlgoritmosComponent é integrado para permitir ao usuário a seleção e parametrização dos algoritmos de otimização a serem executados como visto na Figura 6. Adjacente a este seletor está o Botão "Traçar Rota", o *trigger* principal que inicia o processo de otimização. Ao ser acionado, este botão comunica de forma assíncrona o conjunto de Pontos de interesse (POIs) e as configurações algorítmicas selecionadas ao *backend* para o processamento da rota. Finalmente, o MetricaComponent tem a responsabilidade de apresentar as métricas de desempenho resultantes das execuções dos algoritmos como mostrado na Figura 7.

Figura 5 – Pontos de interesse selecionados na aplicação



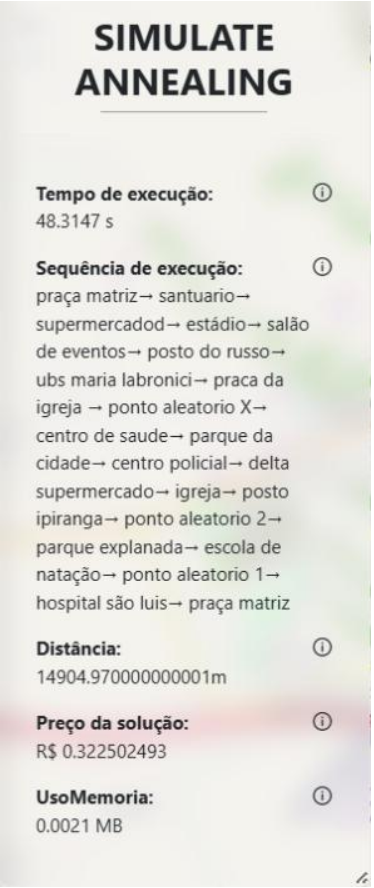
Fonte: autoria própria

Figura 6 – Componente de selecionar algoritmos



Fonte: autoria própria

Figura 7 – Exibição de resultados e métricas



Fonte: autoria própria

A entrada de dados e a seleção de novos POIs são gerenciadas através de *pop-ups* modais, visando um processo de *input* focado e não disruptivo. Ao clicar em “adicionar pontos” no *sidebar*, o sistema oferece ao usuário duas modalidades de seleção. A primeira, gerenciada pelo HistoricoComponent, foca na reutilização de dados, permitindo a seleção de endereços previamente salvos ou utilizados em rotas anteriores. A segunda, operacionalizada pelo MapInputComponent, permite a definição de novos locais. Nesta modalidade, o usuário pode interagir diretamente com o mapa para escolher o ponto de interesse e depois nomeá-lo. Após a validação no modal, as coordenadas do POI adicionado são persistidas na lista de pontos.

4.7.5 Fluxo de Dados da UI: O Caso de Uso de Cálculo de Rota

O fluxo de interação do usuário para calcular uma rota exemplifica a arquitetura reativa. Ao inicializar, o StartUpService busca os POIs e algoritmos disponíveis na API

(via RoutzAPI) e atualiza os *Signals* nos *Stores* (PoiStoreService, AlgoritmoStoreService). Os componentes (MapComponent, SelecionadorDeAlgoritmosComponent) reagem e exibem os dados iniciais. Quando o usuário clica em adicionar pontos, e seleciona pontos no mapa ou no histórico, o MapComponent notifica o PoiStoreService, que atualiza o *Signal* pontosSelecionados. O ListaDePontosComponent reage a essa mudança e atualiza a lista na *sidebar*.

Após a seleção dos algoritmos no *dropdown*, o usuário clica em "Calcular Rota". A SideBarComponent lê os *Signals* de estado (pontosSelecionados, algoritmoSelecionado), monta o AlgoritmosDto e aciona a requisição para o RoutzAPI, ativando o *pop-up* de carregamento. Após o processamento no *backend*, a resposta é recebida. O *pop-up* é desativado, e a resposta é passada ao GerenciaAlgoritmoResService, que atualiza os *Signals* de resultado (rota, métricas) no AlgoritmoStoreService. Por último, os componentes reagem ao resultado. O MapComponent desenha a nova rota e o MetricasComponent exibe as métricas de desempenho nos respectivos algoritmos.

4.8 BANCO DE DADOS

A escolha do banco de dados foi estratégica para a performance e expressividade do sistema, optando-se pelo Neo4j devido à sua natureza de grafos, ideal para o Problema do Caixeiro Viajante (TSP). Além disso quando adicionado com o *plugin* do APOC, ganha-se um conjunto de *procedures* já implementadas que facilitam em alguns casos de uso, exemplo de isso é a função que já retorna o menor caminho entre dois pontos bastando apenas usar *apoc.djsktra*

4.8.1 Justificativa da Escolha: A Vantagem Nativa dos Grafos

A decisão de utilizar o Neo4j, um banco de dados nativo orientado a grafos, fundamenta-se na sua superioridade em relação a modelos relacionais para representar e consultar dados altamente conectados, como redes viárias. Enquanto bancos relacionais exigem JOINs complexos e frequentemente ineficientes para travessias de múltiplos níveis (essenciais em problemas de rota), o Neo4j armazena relacionamentos como entidades de primeira classe. Isso permite consultas de caminho e vizinhança com uma velocidade e simplicidade incomparáveis, crucial para

calcular as distâncias reais entre os pontos de interesse selecionados pelo usuário de forma performática.

4.8.2 Modelo de Dados: Nós e Relacionamentos

O modelo é composto por duas entidades principais, ou "nós", que representam os objetos físicos no mapa. A primeira entidade é o POI (Ponto de Interesse), que representa locais específicos e nomeados adicionados pelos usuários, como "praça matriz" ou "restaurante". Estes nós servem como pontos de partida ou destino e armazenam atributos essenciais como id, latitude, longitude, *name* e um *osmid* opcional para referência externa.

A segunda entidade é o *Node* (Nó da Rede Viária). Estes nós representam os pontos de junção, interseção ou cruzamentos da rede viária. Eles funcionam como os vértices da malha de ruas, permitindo a navegação e a transição entre diferentes segmentos. Cada Node possui um id único e coordenadas geográficas implícitas que são cruciais para calcular as distâncias das ruas que se conectam a ele.

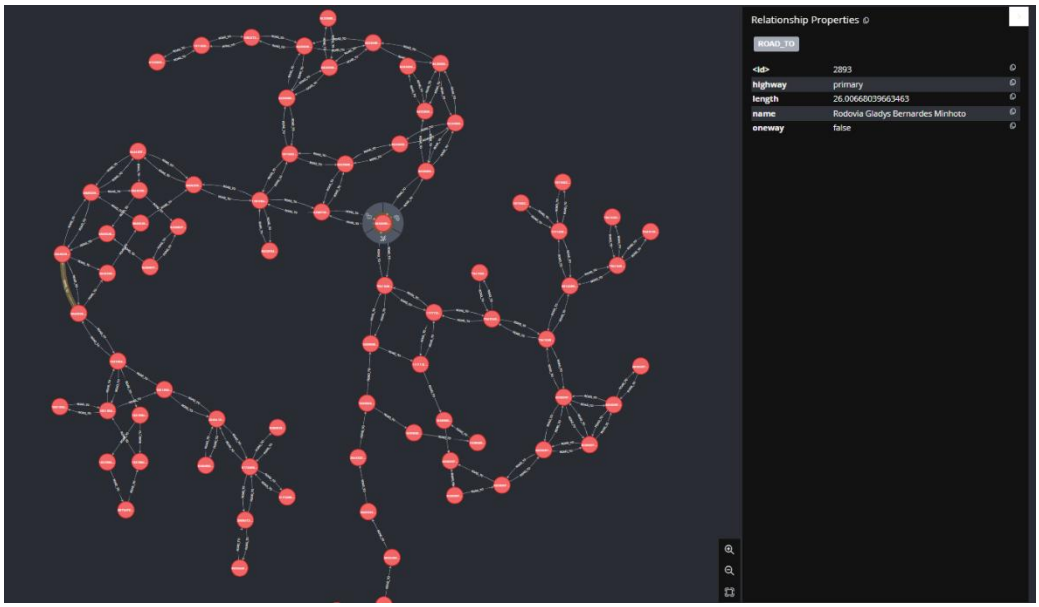
Para conectar essas entidades e formar a lógica da rede, utilizamos dois tipos principais de relacionamentos, ou "arestas". O relacionamento mais crítico para a navegação é o ROAD_TO, que representa os segmentos de via ou as próprias ruas. Esta aresta conecta duas entidades Node (cruzamentos), indicando que existe um caminho direto entre elas. É neste relacionamento que armazenamos as propriedades da via, como *name, length, oneway e highway*.

O segundo relacionamento é o CONNECTED_TO. Sua função é "ancorar" um POI (que existe em coordenadas livres) ao nó da rede viária (Node) mais próximo. Isso é vital, pois para iniciar ou terminar uma rota em um POI, o sistema deve primeiro encontrar o ponto de acesso correspondente na malha de ruas. Esta aresta conecta um (POI) a um (Node) e pode, opcionalmente, armazenar a distância exata entre os dois pontos.

Com esta estrutura, o processo de cálculo de rota torna-se claro. Para encontrar um caminho entre dois POI, o algoritmo primeiro localiza os nós Node de partida e destino (seguindo os relacionamentos CONNECTED_TO de cada POI). Em seguida, ele executa um algoritmo de busca de caminho, como Dijkstra ou A*, navegando exclusivamente pela rede de relacionamentos ROAD_TO para encontrar o caminho ideal entre os dois cruzamentos.

Na Figura 8 é exemplificado como é organizado e relacionado as entidades no banco de dados

Figura 8 – Exemplificando relacionamento de entidades no banco Neo4j



Fonte: autoria própria

4.8.3 O Papel Estratégico da Biblioteca APOC: Computação no Banco

É na integração com a biblioteca APOC (*Awesome Procedures on Cypher*) que a escolha pelo Neo4j demonstra seu maior valor estratégico, transformando o banco de dados de um mero repositório em um poderoso motor computacional para análise de grafos. Seria impraticável carregar grandes porções do grafo para a memória da aplicação (RoutezAPI) para calcular caminhos mais curtos.

A biblioteca APOC, ao estender a linguagem *Cypher* com procedimentos otimizados que rodam diretamente dentro do motor do Neo4j, permite delegar essa tarefa complexa. Especificamente, o DistanceMatrixService invoca procedimentos como apoc.algo.dijkstra para encontrar a rota de menor custo entre cada par de pontos, utilizando a propriedade distancia como peso.

Esta abordagem tem eficiência e performance ao manter a computação intensiva em grafos dentro do banco otimizado para isso, e o código da aplicação (DistanceMatrixService) permanece limpo e desacoplado, focado apenas em orquestrar as chamadas APOC e formatar os resultados na matriz de adjacência

(numpy.ndarray). A sinergia entre o modelo nativo do Neo4j e o poder computacional da APOC é, portanto, um fator habilitador essencial para a performance e escalabilidade da RoutezAPI.

5 CONFIGURAÇÃO DOS EXPERIMENTOS

O presente capítulo descreve detalhadamente a configuração dos experimentos utilizados para avaliar o desempenho de diferentes algoritmos aplicados ao PCV. O objetivo é apresentar os parâmetros de cada algoritmo, o ambiente de execução e os critérios adotados, garantindo que os testes sejam reprodutíveis e comparáveis.

5.1 PARÂMETROS DOS ALGORITMOS

Neste capítulo, são apresentados os algoritmos utilizados, algoritmo genético, *IQAOA*, força bruta e *simulated annealing* juntamente com suas configurações específicas. Cada algoritmo é detalhado em termos de parâmetros, ambiente de execução e critérios de avaliação, permitindo compreender seu funcionamento e servindo de base para a análise dos resultados nos experimentos.

O algoritmo genético foi configurado com uma população composta por 100 indivíduos e executado ao longo de 500 gerações. Esses valores foram definidos com o objetivo de equilibrar a diversidade populacional e a profundidade da busca no espaço de soluções. O processo evolutivo empregou operadores clássicos de seleção por torneio, cruzamento por ordenação e mutação por troca, aplicados com uma taxa de mutação de 2% e tamanho de torneio igual a 5.

Além disso, foi adotada uma estratégia de elitismo, garantindo que o melhor indivíduo de cada geração fosse preservado na próxima. Esses parâmetros foram ajustados empiricamente de forma a promover um equilíbrio entre exploração (diversificação) durante o processo de otimização e podem ser visualizados na Tabela 1

Tabela 1 - Parâmetros de configuração do algoritmo genético

Algoritmo	**Gerações**	**População**	**Taxa de mutação**	**indivíduos para torneio**
Algoritmo genético	500	100	2%	5

Fonte: autoria própria

O IQAOA foi configurado para execução em modo híbrido, no qual a otimização dos parâmetros variacionais é realizada em um simulador clássico local, enquanto a amostragem final pode ser conduzida em um dispositivo quântico real (QPU) da AWS *braket*.

Os parâmetros característicos da execução de sistemas quânticos utilizado nas plataformas atuais (dada a data de execução deste trabalho) foram definidos conforme descrito na Tabela 2, adotando uma repetição (reps = 1) devido à simplificação estrutural do circuito quântico, que reduz a necessidade de camadas profundas, assim podendo ser otimizado mais facilmente. O otimizador clássico *constrained optimization by linear approximations* (COBYLA) foi selecionado por sua robustez em problemas variacionais não lineares e sua eficiência em espaços de busca de baixa dimensão. O número máximo de iterações do otimizador foi fixado em 50, garantindo convergência estável sem aumento excessivo de custo computacional (FERNÁNDEZ 2022).

Para a etapa de amostragem, foram utilizados 2.000 *shots (amostras)*, permitindo obter uma estimativa estatisticamente relevante das probabilidades dos estados quânticos e minimizando o erro estatístico (EGGERT 2023). Essa configuração busca equilibrar a fidelidade da solução quântica e a eficiência da otimização clássica, proporcionando uma execução estável do IQAOA em ambiente híbrido.

Tabela 2 - Parâmetros de configuração do algoritmo IQAOA

Algoritmo	**Reps**	**Otimizado**	***Shots***	**Iterações máximas**
IQAOA	1	Cobyla	2000	50

Fonte: autoria própria

O *simulated annealing* foi configurado com temperatura inicial de 10.000, taxa de resfriamento de 0,995 e 100 iterações por temperatura. Esses parâmetros foram definidos com base em testes iniciais para aprendizado do funcionamento da plataforma de modo a equilibrar a exploração do espaço de soluções e a convergência eficiente para ótimos locais.

A alta temperatura inicial permite uma maior aceitação de soluções piores nas etapas iniciais, promovendo uma busca mais ampla, enquanto a taxa de resfriamento

próxima de 1,0 garante uma redução gradual da temperatura, preservando a capacidade de escapar de mínimos locais até as últimas iterações.

O número de iterações por temperatura foi escolhido para assegurar estabilidade estatística no processo de amostragem e refinamento de soluções. Essa configuração foi projetada para atingir um compromisso entre qualidade da rota obtida e tempo total de execução, conforme resumido na Tabela 3.

Tabela 3 - Parâmetros de configuração do algoritmo *simulated annealing*

Algoritmo	Temperatura inicial	Taxa de resfriamento	Iterações máximas
Simulated annealing	10000	0,995	100

Fonte: autoria própria

O de força bruta não possui parâmetros de configuração, servindo unicamente como referência (*baseline*) para validar os resultados obtidos pelos outros métodos.

5.2 AMBIENTE DE EXECUÇÃO

Os experimentos foram conduzidos em dois ambientes distintos, garantindo a possibilidade de comparar o desempenho clássico e quântico do IQAOA.
No ambiente clássico (simulador), foram utilizados os seguintes recursos, processador (CPU) i7-12650H (2.30 GHz), memória RAM 32GB e sistema operacional Windows 11.

No ambiente quântico, o provedor de *hardware* utilizado foi AWS *Braket*, na máquina quântica Rigetti's Ankaa-3, com 84 qubits disponíveis. Foram consideradas as restrições de fila e o custo de acesso como parte da análise, de forma a refletir condições realistas de execução.

5.3 CRITÉRIOS DE EXECUÇÃO

Cada algoritmo foi executado uma única vez em cada cenário experimental. Os resultados foram avaliados com base em quatro critérios principais: tempo de execução, distância das rotas, custo computacional e custo monetário.

O tempo de execução compreende tanto o processamento clássico quanto, quando aplicável, a latência associada ao uso de *hardware* quântico. A distância das

rotas representa a qualidade da solução obtida, indicando o menor percurso encontrado pelo algoritmo. O custo computacional envolve o consumo de recursos de *hardware* (CPU e QPU), enquanto o custo monetário considera o valor estimado de execução, especialmente relevante para tarefas executadas em provedores de nuvem como a AWS *Braket*.

5.4 PONTOS DE INTERESSE OU DESTINOS

Os conjuntos de pontos de interesse serviram como base para a análise combinatória, sendo representados por coordenadas geográficas (latitude e longitude). Para o caso de teste 1, foram considerados 5 pontos: P1 - Praça Matriz, P2 - Parque Explanada, P3 - Hospital São Luis, P4 - Estádio e P5 - Posto Ipiranga, cujas coordenadas estão detalhadas na Tabela 4.

Tabela 4 - Coordenadas geográficas de 5 destinos

Ponto	**Latitude**	**Longitude**
P1 - Praça Matriz	-23.284009075594128	-47.673733234405525
P2 - Parque Explanada	-23.272694890268852	-47.67888307571411
P3 - Hospital são Luis	-23.289085217069484	-47.68066406250001
P4 - Estádio	-23.28298198658816	-47.667253017425544
P5 – Posto Ipiranga	-23.27196541690759	-47.67386198043824

Fonte: autoria própria

Para o caso de teste 2, foram considerados 10 pontos, incluindo os anteriores e mais cinco adicionais: P6 - Parque da Cidade, P7 - Escola de Natação, P8 - Igreja, P9 - Salão de Eventos e P10 - Centro Policial, conforme apresentado na Tabela 5. Estes pontos de interesse constituem as instâncias utilizadas para avaliar o desempenho dos algoritmos nos experimentos.

Tabela 5 - Coordenadas geográficas de 10 destinos

Ponto	**Latitude**	**Longitude**
P1 - Praça Matriz	-23.284009075594128	-47.673733234405525
P2 - Parque Explanada	-23.272694890268852	-47.67888307571411
P3 - Hospital são Luis	-23.289085217069484	-47.68066406250001
P4 - Estádio	-23.28298198658816	-47.667253017425544

Ponto	Latitude	Longitude
P5 – Posto Ipiranga	-23.27196541690759	-47.67386198043824
P6 – Parque da Cidade	-23.29046585582683	-47.67217755317689
P7 – Escola de Natação	-23.27733901656989	-47.67804622650147
P8 – Igreja	-23.276656498945176	-47.673711776733406
P9 – Salão de Eventos	-23.280239813545755	-47.663755416870124
P10 – Centro Policial	-23.287441127214283	-47.67468810081482

Fonte: autoria própria

Para o caso de teste 3, foram considerados 20 pontos de interesse, englobando os 10 pontos anteriores e mais dez adicionais: P11 - Ponto Aleatório 1, P12 - Ponto Aleatório 2, P13 - Praça da Igreja, P14 - Santuário, P15 - Centro de Saúde, P16 - Ponto Aleatório 3, P17 - Supermercado, P18 - Delta Supermercado, P19 - Posto do Russo e P20 - UBS Maria Labronici como mostra a Tabela 6. Esses pontos constituem as instâncias utilizadas nos experimentos, permitindo avaliar o desempenho dos algoritmos frente a um maior número de destinos e cenários mais complexos.

Tabela 6 - Coordenadas geográficas de 20 destinos

Ponto	Latitude	Longitude
P1 - Praça Matriz	-23.284009075594128	-47.673733234405525
P2 - Parque Explanada	-23.272694890268852	-47.67888307571411
P3 - Hospital são Luís	-23.289085217069484	-47.68066406250001
P4 - Estádio	-23.28298198658816	-47.667253017425544
P5 – Posto Ipiranga	-23.27196541690759	-47.67386198043824
P6 – Parque da Cidade	-23.29046585582683	-47.67217755317689
P7 – Escola de Natação	-23.27733901656989	-47.67804622650147
P8 – Igreja	-23.276656498945176	-47.673711776733406
P9 – Salão de Eventos	-23.280239813545755	-47.663755416870124
P10 – Centro Policial	-23.287441127214283	-47.67468810081482
P11 - Ponto aleatório 1	-23.28117312356102	-47.68205239971965
P12 - Ponto aleatório 2	-23.272031093268335	- 47.67723386684169
P13 - Praça da igreja	-23.284730029025784	-47.667171282945944
P14 – santuário	-23.280809639767693	-47.6706257055307
P15 - Centro de saúde	- 23.28664996211714	-47.67131276571181
P16 - Ponto aleatório 3	-23.285628656191804	-47.66800782460922
P17 - supermercado	-23.283861075239813	-47.67020182320487

Ponto	Latitude	Longitude
P18 - delta supermercado	-23.28202009716208	-47.674742353837665
P19 - posto do russo	-23.28299355446011	- 47.66043916704392
P20 - Ubs Maria Labronici	- 23.288298619370483	- 47.6618878018047

Fonte: autoria própria.

6 VALIDAÇÃO E RESULTADOS

A validação e resultados tem como objetivo apresentar e analisar os resultados obtidos a partir da aplicação de diferentes algoritmos ao PCV um problema clássico de otimização combinatória, cuja meta é determinar a rota mais eficiente para visitar múltiplos destinos com o menor custo total possível.

Neste estudo, foram implementadas e comparadas quatro abordagens distintas de solução:

- Força bruta, representando os métodos clássicos determinísticos;
- Algoritmo genético, como exemplo de técnica metaheurística evolutiva;
- *IQAOA*, representando a abordagem quântica híbrida;
- *Simulated annealing*, considerado um método metaheurístico clássico baseado no processo de resfriamento de metais.

Os resultados são inicialmente apresentados com base nas execuções realizadas no sistema de testes, abrangendo o comportamento individual de cada algoritmo e suas respectivas características de desempenho. Em seguida, é feita uma comparação entre as abordagens, destacando diferenças quanto à eficiência e precisão das soluções obtidas.

Posteriormente, o foco recai sobre o IQAOA, analisado em diferentes ambientes de execução (*hardware* clássico e quântico) a fim de observar possíveis variações de desempenho. Por fim, realiza-se uma discussão geral dos resultados, sintetizando as principais observações e conclusões obtidas a partir da análise experimental.

6.1 APRESENTAÇÃO DOS RESULTADOS NO SISTEMA DE TESTES

Nesta seção são apresentados os resultados obtidos a partir da execução dos algoritmos implementados para resolver o PCV. O objetivo é fornecer uma visão detalhada do desempenho de cada abordagem, considerando métricas como custo total da rota, tempo de execução e eficiência na obtenção de soluções. Para facilitar a análise, os resultados foram organizados em duas perspectivas: a comparação entre os diferentes algoritmos e a avaliação do IQAOA em ambientes de execução distintos.

6.1.1 Comparação entre os algoritmos

Nesta subseção, os resultados obtidos pelos quatro algoritmos implementados força bruta, algoritmo genético, IQAOA em *hardware* clássico e *simulated annealing* são apresentados e analisados.

6.1.1.1 Comparação com 5 pontos

Para o teste de comparação com 5 pontos de destino, foram utilizadas as coordenadas apresentadas na Tabela 4, do capítulo de configuração dos experimentos, resultando em uma única rota, ilustrada na Figura 9.

Figura 9 - Ferramenta web rota para 5 destinos

Fonte: autoria própria.

Figura 10 - Ferramenta web resultado dos algoritmos com 5 destinos

Fonte: autoria própria.

Tabela 7 - Resultados dos algoritmos com 5 destinos

Algoritmo	Tempo de execução (s)	Distância total (m)	Custo computacional (mb)
IQAOA (quântico)	5,739	6940,59	-
Algoritmo genético	2,0042	6940,59	0,0289
Força bruta	0,0012	6940,59	0,0011
Simulated annealing	19,888	6940,59	0,0011

Fonte: autoria própria.

A análise dos resultados da Tabela 7, por meio da Figura 10 apresenta uma comparação de desempenho entre o algoritmo quântico aproximado de otimização (IQAOA) e os algoritmos clássicos de Algoritmo Genético, Força Bruta e *simulated annealing* na resolução do problema do caixeiro viajante (PCV) para uma instância de cinco destinos. Para este cenário de pequena escala, notou-se que todos os algoritmos convergiram para a mesma distância total de 6.940,59 metros. Este resultado demonstra que as diferenças de desempenho entre os métodos se manifestaram estritamente nas métricas de tempo de execução e custo computacional.

O campo referente ao custo computacional do IQAOA foi designado como não aplicável. Isso decorre do fato de que a execução do IQAOA ocorreu em um ambiente de computação quântica na nuvem *AWS Braket*. Neste contexto, a métrica de custo não se traduz em consumo de memória RAM (MB), como é o padrão para os

algoritmos clássicos. O custo em sistemas quânticos é intrinsecamente ligado à utilização de recursos quânticos específicos, como o número de qubits, a profundidade do circuito e o tempo de acesso à *backend* quântica.

Considerando o pequeno número de destinos, o algoritmo de força bruta demonstrou ser a solução mais eficiente para esta instância de baixa complexidade. Este algoritmo apresentou o menor tempo de execução (0,0012 s) e um dos menores custos computacionais (0,0011 MB). Tal equilíbrio entre a eficiência temporal e o baixo custo computacional valida o algoritmo de força bruta como a solução mais adequada para problemas de otimização de pequena escala.

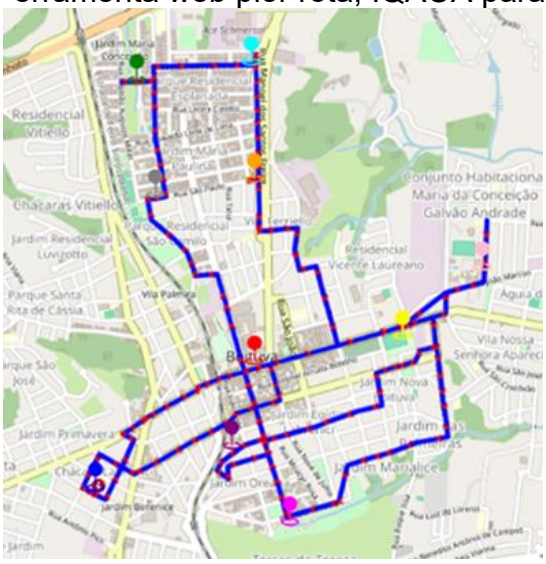
6.1.1.2 Comparação com 10 pontos

Para o teste de comparação com 10 pontos de destino, foram utilizadas as coordenadas apresentadas na Tabela 5, do capítulo de configuração dos experimentos. Este cenário apresentou maior complexidade, permitindo observar o comportamento de cada algoritmo em situações mais desafiadoras. Os algoritmos conseguiram encontrar a melhor rota para os 10 destinos, ilustrada na Figura 11, enquanto o IQAOA gerou uma rota alternativa, considerada a pior para este conjunto de pontos, apresentada na Figura 12.

Figura 11 - Ferramenta web melhor rota para 10 destinos

Fonte: autoria própria.

Figura 12 - Ferramenta web pior rota, IQAOA para 10 destinos



Fonte: autoria própria.

Figura 13 - Ferramenta web resultado dos algoritmos com 10 destinos

SIMULATE ANNEALING

Tempo de execução:
43.0428 s

Sequência de execução:
praça matriz→ estádio→ salão de eventos→ igreja→ posto ipiranga→ parque explanada→ escola de natação→ hospital são luis→ centro policial→ parque da cidade→ praça matriz

Distância:
10165.18m

Preço da solução:
R$ 0.287319510

UsoMemoria:
0.0014 MB

FORÇA BRUTA

Tempo de execução:
30.6832 s

Sequência de execução:
praça matriz→ estádio→ salão de eventos→ igreja→ posto ipiranga→ parque explanada→ escola de natação→ hospital são luis→ centro policial→ parque da cidade→ praça matriz

Distância:
10165.18m

Preço da solução:
R$ 0.204841504

UsoMemoria:
0.0014 MB

Genêtico

Tempo de execução:
4.3370 s

Sequência de execução:
igreja→ posto ipiranga→ parque explanada→ escola de natação→ hospital são luis→ centro policial→ parque da cidade→ praça matriz→ estádio→ salão de eventos→ igreja

Distância:
10165.179999999998m

Preço da solução:
R$ 0.029042689

UsoMemoria:
0.0372 MB

IQAOA Quântico

Tempo de execução:
6,122 s

Sequência de execução:
praça matriz→ hospital são luis→ estádio→ parque da cidade→ escola de natação→ parque explanada→ posto ipiranga→ igreja→ salão de eventos→ centro policial→ praça matriz

Distância:
13333.59m

Preço da solução:
R$ 1.602000000

Qtd. de qubits no circuito:
10

Fonte: autoria própria.

Tabela 8 - Resultados dos algoritmos com 10 destinos

Algoritmo	**Tempo de execução**	**Distância total**	**Custo computacional**
IQAOA (quântico)	6,122	13333,59	–
Algoritmo Genético	4,337	10165,18	0,0372
Força Bruta	30,6832	10165,18	0,0014
Simulated Annealing	43,0428	10165,18	0,0014

Fonte: autoria própria.

A análise dos resultados da Tabela 8, por meio da Figura 13 - Ferramenta web resultado dos algoritmos com 10 destinos com seus dados consolidados, demonstra que, para o conjunto de 10 destinos utilizado, as diferenças entre os algoritmos ocorreram nos tempos de execução, custos computacionais e na distância total das rotas. Com o aumento do número de pontos em relação ao experimento anterior, tornou-se evidente que a partir de 10 destinos os algoritmos passam a apresentar variações significativas em suas performances, gerando múltiplas rotas possíveis. O algoritmo genético destacou-se como o de melhor desempenho geral, equilibrando tempo de execução e qualidade da solução, enquanto o IQAOA, embora competitivo, não superou o algoritmo genético nesse conjunto específico de destinos, evidenciando a diferença de comportamento entre abordagens clássicas e quânticas em problemas de média escala.

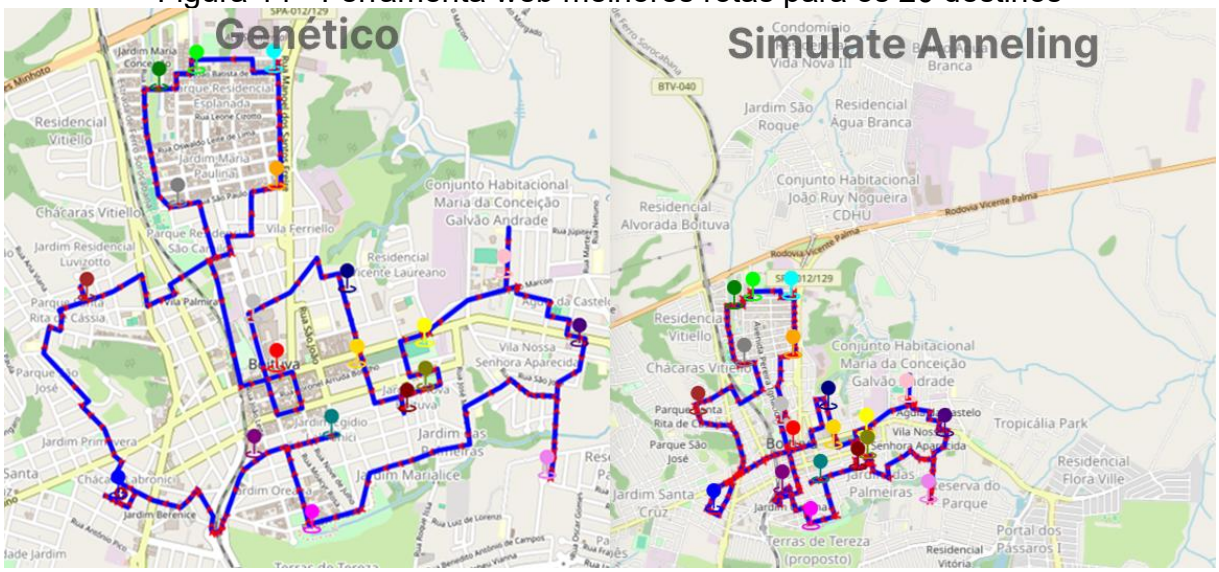
O força bruta apresentou desempenho inferior devido à sua complexidade exponencial, que aumenta drasticamente o tempo de execução à medida que o número de destinos cresce. Já o *simulated annealing*, embora capaz de escapar de mínimos locais, teve desempenho inferior por apresentar maior tempo de execução e variação na distância total, resultado da dependência de seus parâmetros de configuração. Dessa forma, é possível observar como a escolha do algoritmo impacta diretamente na eficiência e viabilidade da solução para problemas de otimização combinatória em diferentes escalas.

6.1.1.3 Comparação com 20 pontos

Para o teste de comparação com 20 pontos de destino, foram utilizadas as coordenadas apresentadas na Tabela 6, do capítulo de configuração dos experimentos. Este cenário constituiu o maior e mais denso conjunto de pontos utilizado no projeto, apresentando complexidade significativamente maior e permitindo avaliar o comportamento dos algoritmos em situações de alta dificuldade combinatória.

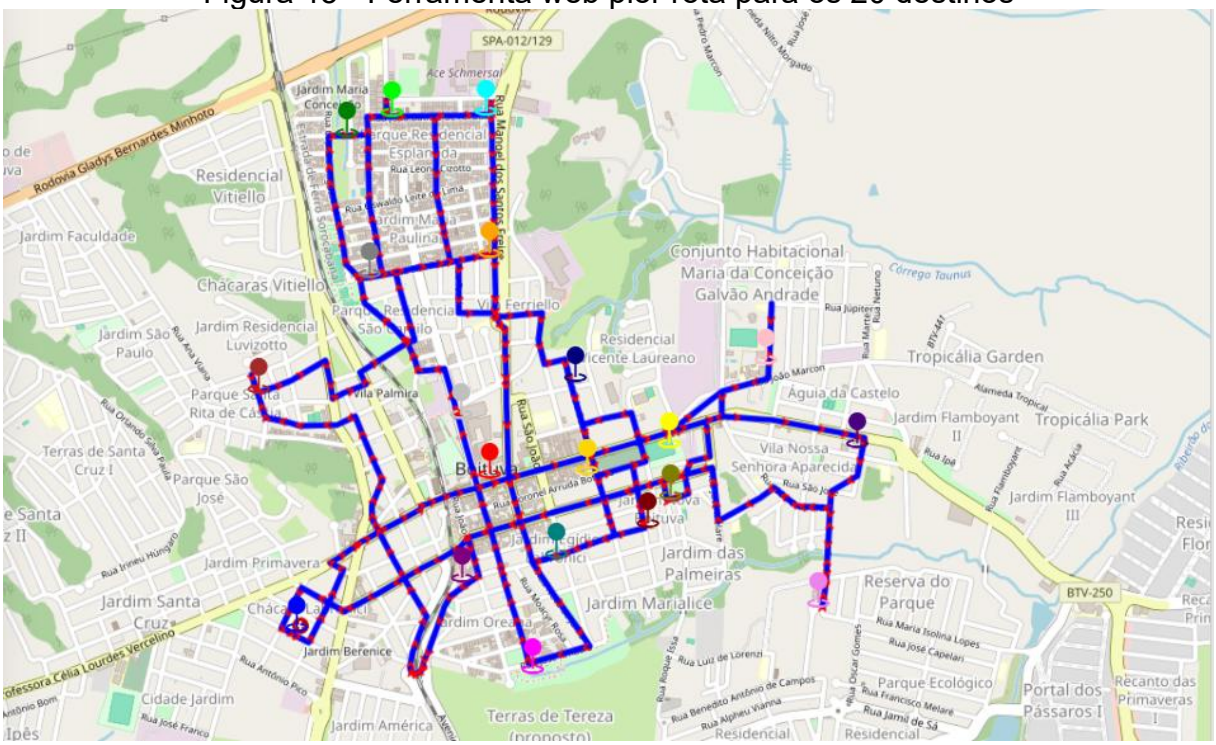
Nesse contexto, o Algoritmo Genético e o *simulated annealing* conseguiram gerar rotas distintas, ilustradas na Figura 14, enquanto o IQAOA apresentou sua rota alternativa, mostrada na Figura 15. A melhor rota em termos de distância total foi obtida pelo *simulated annealing*, destacando sua capacidade de encontrar soluções competitivas mesmo em conjuntos grandes e complexos. O algoritmo de Força Bruta não conseguiu produzir uma rota devido ao tempo de execução impraticável, estimado em aproximadamente 1.680 anos, evidenciando as limitações de métodos exatos em problemas de grande escala.

Figura 14 - Ferramenta web melhores rotas para os 20 destinos



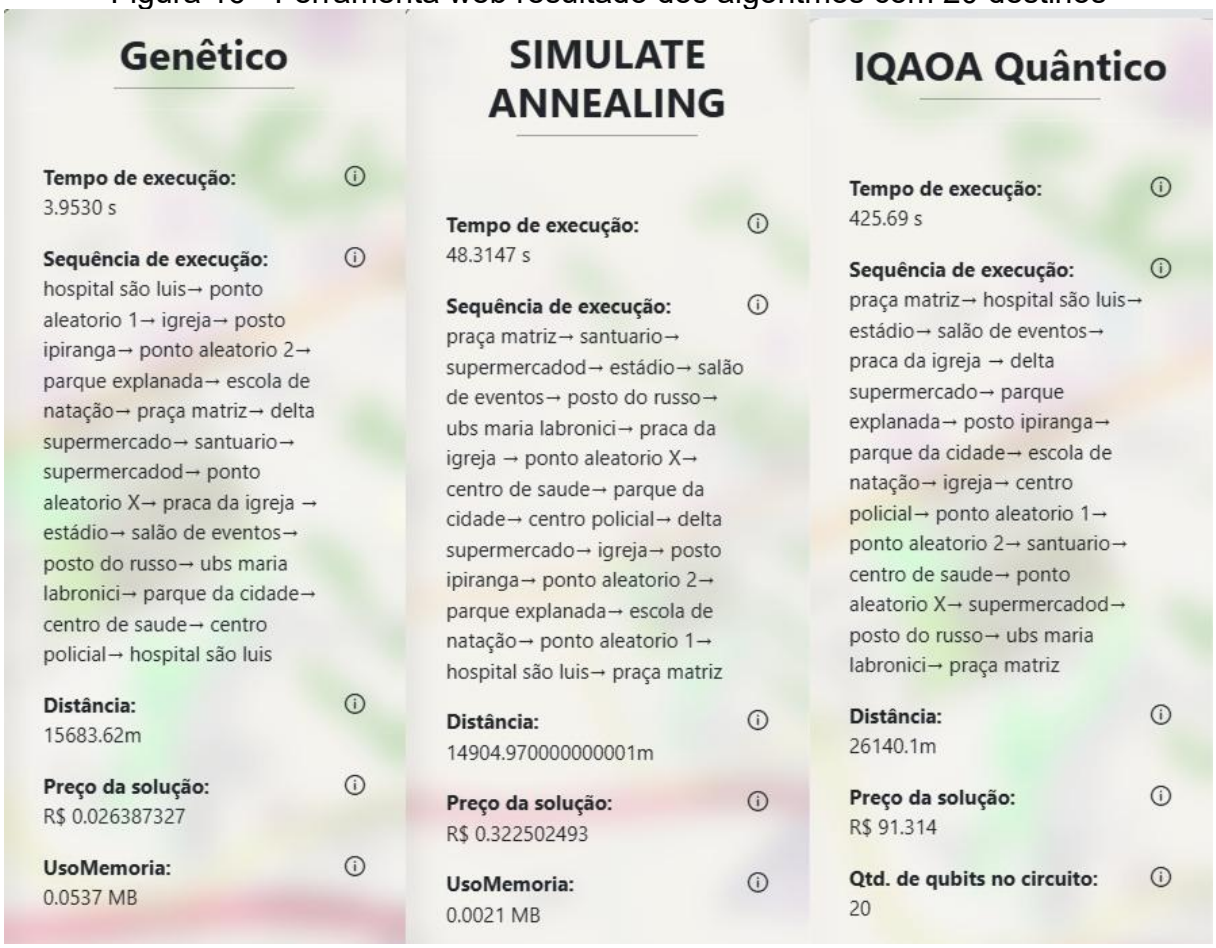
Fonte: autoria própria.

Figura 15 - Ferramenta web pior rota para os 20 destinos



Fonte: autoria própria.

Figura 16 - Ferramenta web resultado dos algoritmos com 20 destinos



Genêtico

Tempo de execução:
3.9530 s

Sequência de execução:
hospital são luis→ ponto aleatorio 1→ igreja→ posto ipiranga→ ponto aleatorio 2→ parque explanada→ escola de natação→ praça matriz→ delta supermercado→ santuario→ supermercadod→ ponto aleatorio X→ praca da igreja → estádio→ salão de eventos→ posto do russo→ ubs maria labronici→ parque da cidade→ centro de saude→ centro policial→ hospital são luis

Distância:
15683.62m

Preço da solução:
R$ 0.026387327

UsoMemoria:
0.0537 MB

SIMULATE ANNEALING

Tempo de execução:
48.3147 s

Sequência de execução:
praça matriz→ santuario→ supermercadod→ estádio→ salão de eventos→ posto do russo→ ubs maria labronici→ praca da igreja → ponto aleatorio X→ centro de saude→ parque da cidade→ centro policial→ delta supermercado→ igreja→ posto ipiranga→ ponto aleatorio 2→ parque explanada→ escola de natação→ ponto aleatorio 1→ hospital são luis→ praça matriz

Distância:
14904.970000000001m

Preço da solução:
R$ 0.322502493

UsoMemoria:
0.0021 MB

IQAOA Quântico

Tempo de execução:
425.69 s

Sequência de execução:
praça matriz→ hospital são luis→ estádio→ salão de eventos→ praca da igreja → delta supermercado→ parque explanada→ posto ipiranga→ parque da cidade→ escola de natação→ igreja→ centro policial→ ponto aleatorio 1→ ponto aleatorio 2→ santuario→ centro de saude→ ponto aleatorio X→ supermercadod→ posto do russo→ ubs maria labronici→ praça matriz

Distância:
26140.1m

Preço da solução:
R$ 91.314

Qtd. de qubits no circuito:
20

Fonte: autoria própria.

Tabela 9 - Resultados dos algoritmos com 20 destinos

Algoritmo	**Tempo de execução (s)**	**Distância total (m)**	**Custo computacional (mb)**
IQAOA (quantico)	425,69	26.140,1	–
Algoritmo genético	3.9530	15.683,62	0,0537
Força bruta	–	–	–
Simulated annealing	48.3147	14.904.97	0.0021

Fonte: autoria própria.

No terceiro caso de teste, composto por 20 pontos, os algoritmos foram avaliados em um cenário no qual o método de força bruta se torna inviável, resultando em aproximadamente $1{,}21645 \times 10^{17}$ possíveis permutações. Considerando um processador Intel Core i7 de 2.30 GHz, capaz de realizar cerca de $2{,}3 \times 10^{9}$ operações por segundo, o tempo estimado para processar todas as combinações seria de aproximadamente 1.680 anos de execução contínua. Nesse contexto, os algoritmos Genético, *simulated annealing* e IQAOA apresentaram comportamentos distintos em termos de desempenho e qualidade das soluções obtidas.

A análise dos resultados da Tabela 9, por meio da Figura 16 com seus dados consolidados, demonstra que, para o conjunto de 20 destinos utilizado, os algoritmos passaram a apresentar valores distintos entre si, incluindo tempo de execução, custo computacional e distância total das rotas. Na ausência da solução por força bruta, que determinaria o ótimo global, a análise comparativa se foca na melhor solução encontrada. O algoritmo *simulated annealing* demonstrou a maior eficácia, identificando a rota mais curta com 14.904,97 m. Embora o Algoritmo Genético tenha sido mais rápido (3,95 s contra 48,31 s), sua solução foi aproximadamente 5,2% menos otimizada, resultando em uma distância de 15.683,62 m. O algoritmo IQAOA, por sua vez, apresentou a solução menos competitiva neste cenário.

6.1.2 Comparação entre ambientes de execução

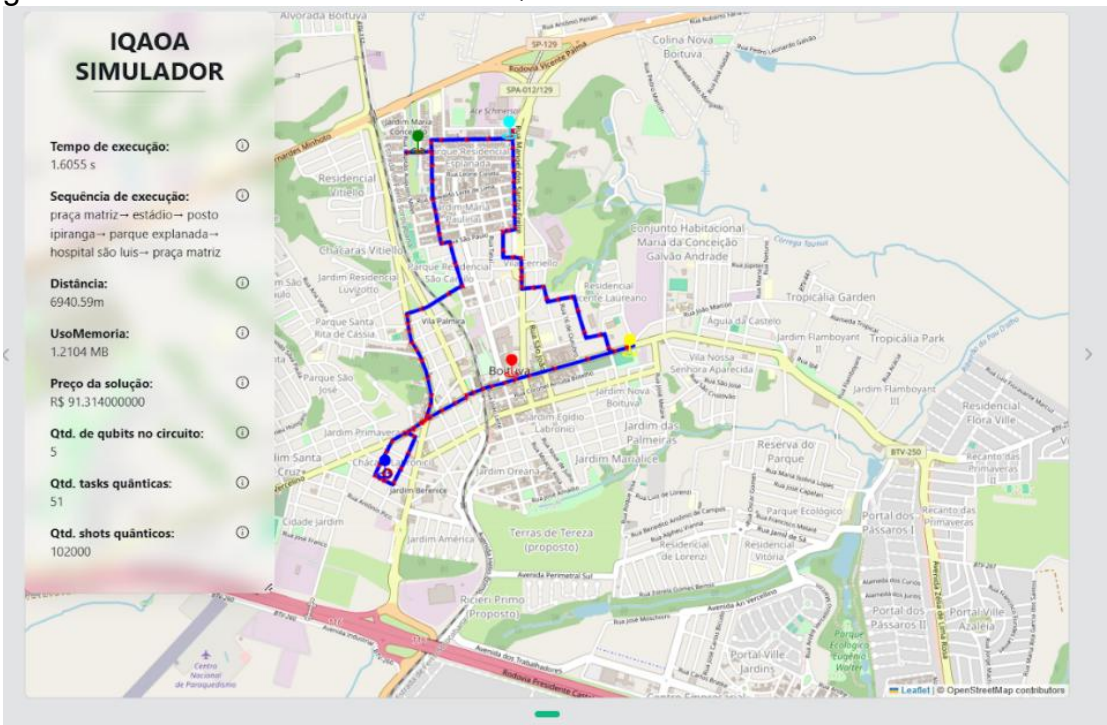
Nesta subseção, o desempenho do IQAOA é avaliado em dois ambientes distintos, simulador clássico e hardware quântico real. O objetivo é observar como as

limitações do ambiente, como capacidade de processamento e ruído quântico, influenciam o comportamento do algoritmo, mesmo em um conjunto reduzido de pontos.

6.1.2.1 Comparação por ambiente com 5 pontos

Utilizando o mesmo conjunto de pontos empregado na comparação por algoritmo, para este caso de teste, o IQAOA foi analisado com cinco pontos. No capítulo de comparação entre os algoritmos, o resultado referente ao ambiente quântico é apresentado na Figura 8, enquanto a Figura 17 a seguir ilustra o desempenho do mesmo algoritmo no ambiente clássico.

Figura 17 - Ferramenta web rota do IQAOA em *hardware* clássico com 5 destinos



Fonte: autoria própria.

Tabela 10 - Resultados do IQAOA em *hardware* clássico e quântico com 5 destinos

Ambiente de execução	Nº de qubits	Tempo de execução	Distância da rota
IQAOA (Simulador clássico)	5	1,6055	6940,59
IQAOA (*Hardware* quântico)	5	5,739	6940,59

Fonte: autoria própria.

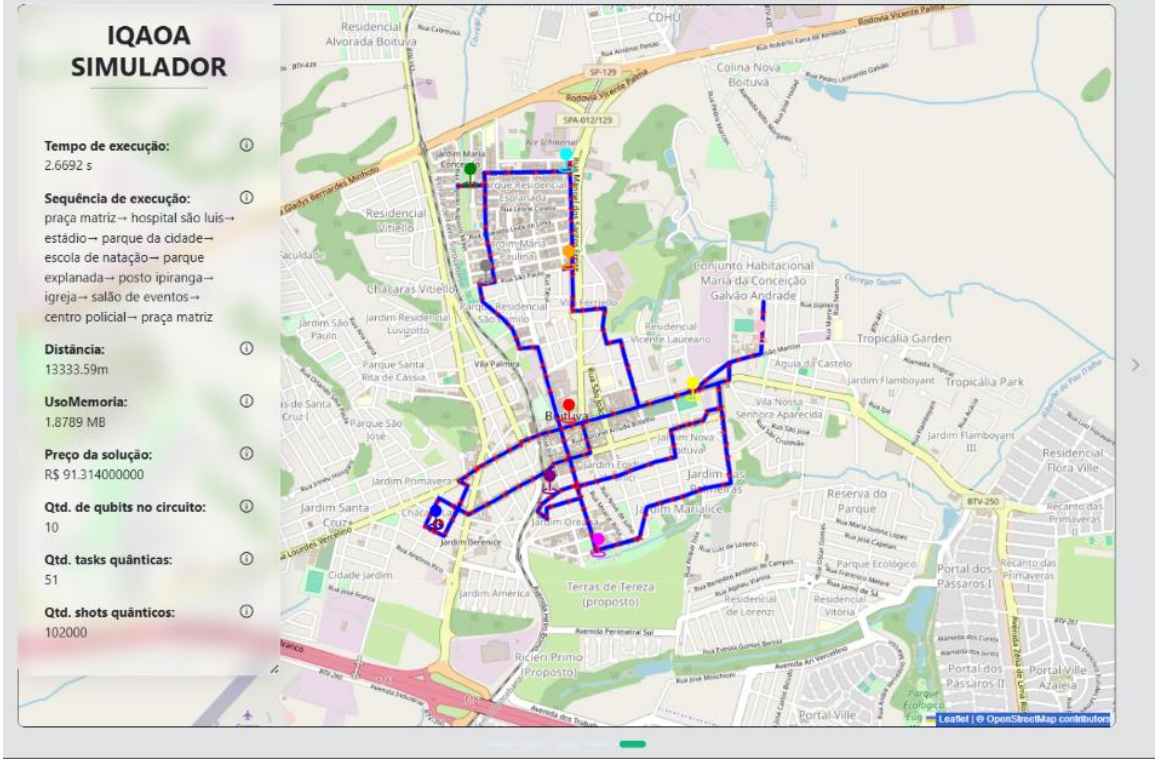
A análise dos resultados, feita por meio da Tabela 10 - Resultados do IQAOA em *hardware* clássico e quântico com 5 destinos, evidencia diferenças importantes entre os ambientes, primordialmente no tempo de execução. Embora ambos os ambientes, o simulador clássico e o *hardware* quântico real, tenham convergido para a mesma distância de rota utilizando 5 qubits, o *hardware* quântico exigiu um tempo de execução significativamente maior em comparação com o simulador clássico. Isso reflete o *overhead* inerente à operação, calibração e mitigação de ruído do *hardware* quântico em tempo real, mesmo na ausência de variações no resultado da otimização para esta instância.

Apesar dessa limitação de eficiência temporal para problemas de pequena escala, o fato de o IQAOA em *hardware* quântico conseguir gerar soluções idênticas ao simulador demonstra sua capacidade de convergência. Esses resultados indicam o potencial promissor do IQAOA em arquiteturas quânticas reais, especialmente para abordagens de otimização mais complexas e de maior escala.

6.1.2.2 Comparação por ambiente com 10 pontos

Utilizando o mesmo conjunto de pontos empregado na comparação por algoritmo, para este caso de teste, o IQAOA foi analisado com dez pontos. No capítulo de comparação entre os algoritmos, o resultado referente ao ambiente quântico é apresentado na Figura 11, enquanto a Figura 18 ilustra o desempenho do mesmo algoritmo no ambiente clássico.

Figura 18 - Ferramenta web rota do IQAOA em *hardware* clássico com 10 destinos



Fonte: autoria própria.

Tabela 11 - Resultados do IQAOA em *hardware* clássico e quântico com 10 destinos

Ambiente de execução	Nº de qubits	Tempo de execução (s)	Distância da rota (m)
IQAOA (Simulador clássico)	10	2,6692	13333,59
IQAOA (*Hardware* quântico)	10	6,122	13333,59

Fonte: autoria própria.

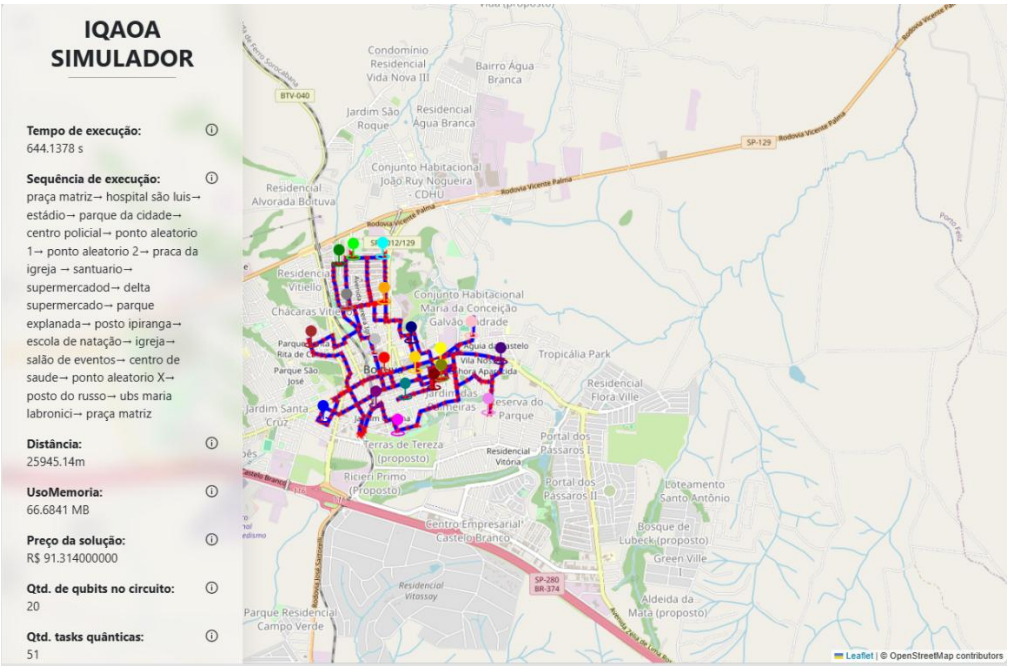
A análise dos resultados para o conjunto de 10 pontos evidencia novamente diferenças entre os ambientes, destacando o tempo de execução como o principal fator. O *hardware* quântico apresentou maior latência devido à operação e calibração em tempo real, enquanto o simulador clássico processou a mesma instância de forma significativamente mais rápida. A validação dessas observações é feita por meio da Tabela 11, que consolida os dados de desempenho de ambos os ambientes.

Apesar dessa diferença temporal, o IQAOA em *hardware* quântico manteve a capacidade de convergir para soluções comparáveis às obtidas no simulador clássico. Esses resultados reforçam a aplicabilidade do algoritmo em ambientes quânticos, apontando sua utilidade para problemas de otimização de média escala, onde a qualidade da solução é mais relevante que o tempo de execução imediato.

6.1.2.3 Comparação por ambiente com 20 pontos

Utilizando o mesmo conjunto de pontos empregado na comparação por algoritmo, para este caso de teste, o IQAOA foi analisado com vinte pontos. No capítulo de comparação entre os algoritmos, o resultado referente ao ambiente quântico é apresentado na Figura 16, enquanto a Figura 19 ilustra o desempenho do mesmo algoritmo no ambiente clássico.

Figura 19 - Ferramenta web rota do IQAOA em *hardware* clássico com 20 destinos



Fonte: autoria própria.

Tabela 12 - Resultados do IQAOA em *hardware* clássico e quântico com 20 destinos

Ambiente de execução	Nº de qubits	Tempo de execução (s)	Distância da rota (m)
IQAOA (Simulador clássico)	20	644.1378	25.945,14
IQAOA (*Hardware* quântico)	20	425.69	26.140,1

Fonte: autoria própria.

A análise dos resultados para o conjunto de 20 pontos evidencia diferenças significativas entre os ambientes, não apenas no tempo de execução, mas também na qualidade das soluções obtidas. Diferente dos cenários anteriores, a Tabela 12 revela que o IQAOA não alcançou a mesma solução nos ambientes simulado e

quântico. O *hardware* quântico apresentou maior velocidade, reduzindo o tempo de execução de 644,1 s no simulador para 425,7 s, representando uma melhoria de aproximadamente 34%.

Contudo, essa vantagem temporal ocorreu ao custo da qualidade da solução. A rota obtida no *hardware* quântico (26.140,1 m) foi ligeiramente inferior à do simulador (25.945,14 m), possivelmente devido à suscetibilidade do *hardware* a ruídos, que pode ter degradado a amostragem e favorecido um caminho não ótimo. Já o simulador, embora mais lento, simula estados ideais de qubits sem a interferência de ruído, garantindo uma solução mais robusta. Esses resultados reforçam a importância de considerar as vantagens e desvantagens entre velocidade e precisão ao utilizar computadores quânticos para problemas de alta complexidade.

6.2 COMPARAÇÃO ENTRE ALGORITMOS E AMBIENTE

Este subcapítulo apresenta a comparação entre os algoritmos Força Bruta, Algoritmo Genético (AG), *IQAOA* e *Annealing*, considerando quatro aspectos principais: tempo de execução, distância da rota, custo computacional e custo monetário. A análise tem como objetivo identificar os pontos fortes e as limitações de cada abordagem, evidenciando como cada algoritmo se comporta em instâncias de diferentes tamanhos do Problema do Caixeiro Viajante (TSP). Dessa forma, busca-se avaliar a aplicabilidade prática de cada técnica, distinguindo aquelas mais adequadas para pequenas instâncias (onde a exatidão é crucial) daquelas que apresentam melhor escalabilidade e eficiência em cenários mais complexos.

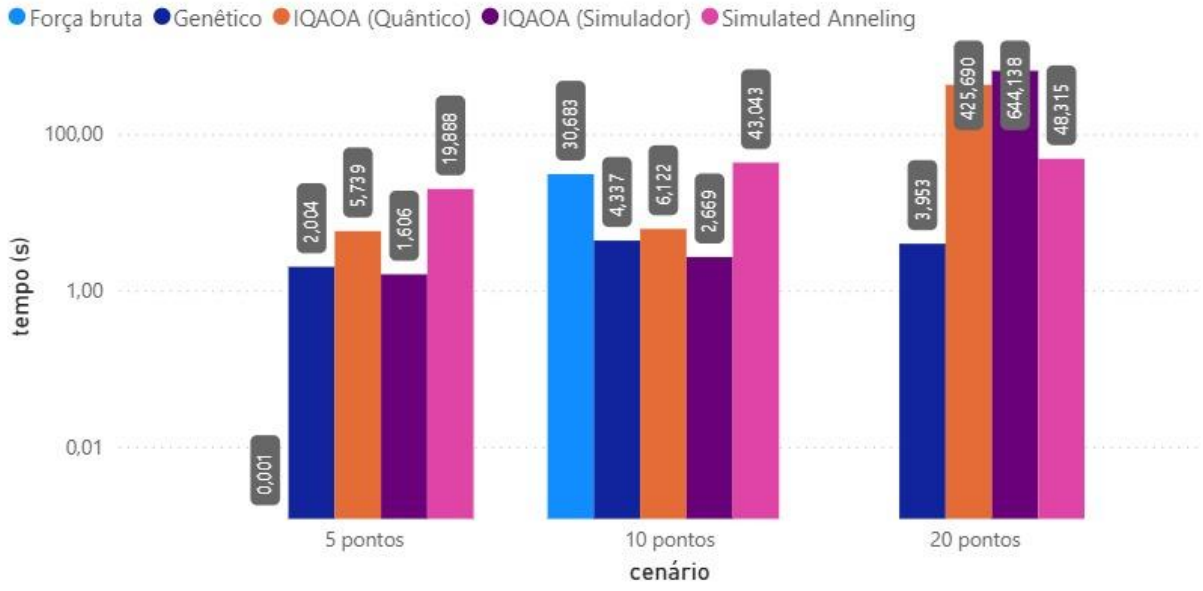
- Força Bruta: serve como *baseline*, fornecendo a solução ótima exata, mas ao custo de elevado tempo de execução e crescimento exponencial da complexidade à medida que o número de pontos aumenta.
- Algoritmo Genético (AG): apresenta bom equilíbrio entre tempo de execução e qualidade da solução, fornecendo resultados próximos ao ótimo em prazos viáveis, mesmo em instâncias maiores.
- *Annealing*: baseado em técnicas inspiradas em recozimento simulado ou quântico, mostra-se competitivo em termos de qualidade da solução, especialmente em instâncias intermediárias, ainda que dependa fortemente da calibração dos parâmetros de resfriamento.

- IQAOA: destaca-se pela inovação, trazendo a perspectiva da computação quântica. Contudo, sua aplicação prática ainda encontra limitações devido ao tempo de execução em *hardware* real, restrições de qubits e impacto do ruído. Testes em simuladores clássicos demonstram resultados promissores e, com a inclusão do custo monetário, será possível avaliar a viabilidade econômica de sua aplicação em comparação com os demais algoritmos.

6.2.1 Tempo de execução

Nesta seção é apresentada a avaliação do desempenho temporal dos algoritmos, evidenciando como o tempo de execução aumenta conforme o número de pontos no PCV. Essa relação é ilustrada na Tabela 13 e na Figura 20, onde o eixo y representa o tempo de execução (em segundos) e o eixo x corresponde aos respectivos cenários de teste.

Figura 20 - Gráfico do tempo de execução dos algoritmos.



Fonte: autoria própria.

Tabela 13 - Tempo de execução dos algoritmos

Pontos	FB (s)	AG (s)	SA (s)	IQAOA clássico (s)	IQAOA quântico (s)
5	0,0012	2,0042	19,888	1,6055	5,739
10	30,6832	4,337	43,0428	2,6692	6,122
20	–	3.9530	48.3147	644.1378	425.69

Fonte: autoria própria.

Com base nos dados da Tabela 13 e na Figura 20, observa-se que o algoritmo de Força Bruta apresenta um aumento significativo no tempo de execução à medida que o número de pontos no PCV cresce. Esse comportamento é esperado, uma vez que o custo computacional da força bruta cresce de forma exponencial.

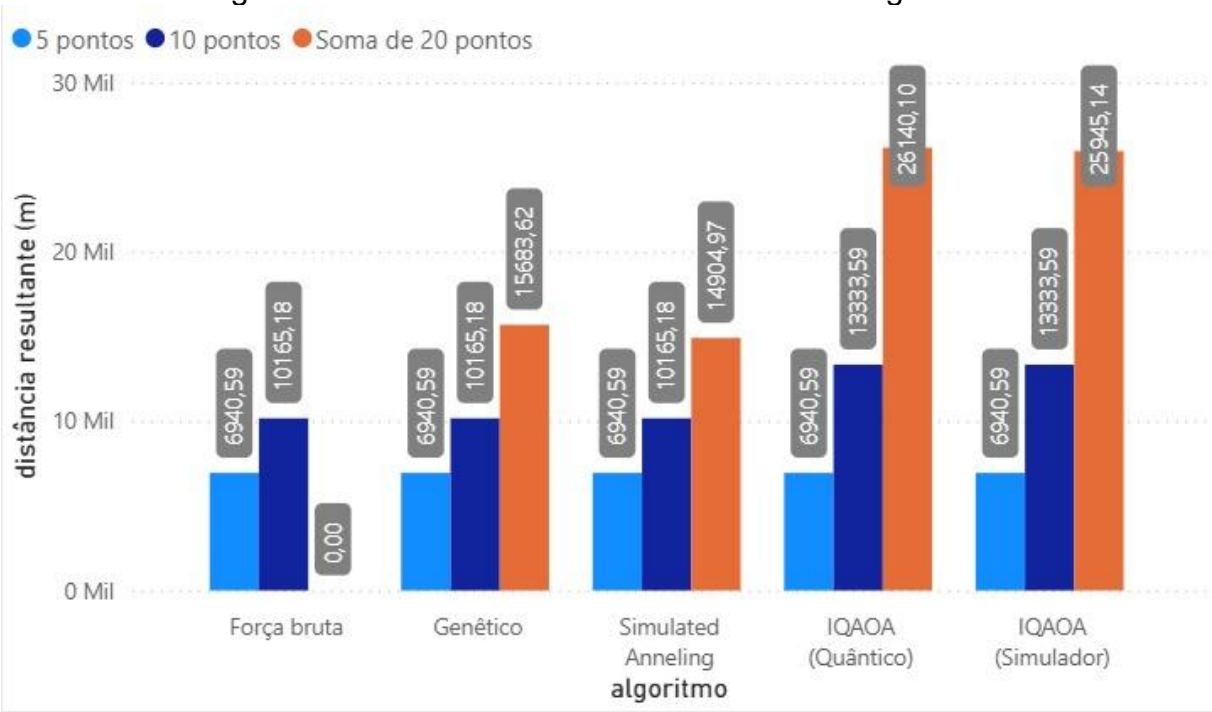
No entanto, considerando que o estudo analisou apenas três cenários de teste (com 5, 10 e 20 pontos), não é possível tirar conclusões definitivas sobre o comportamento assintótico ou a escalabilidade de cada algoritmo. Testes adicionais, envolvendo um número maior de pontos, seriam necessários para compreender melhor como o aumento da complexidade impacta o tempo de execução e para identificar quais algoritmos conseguem manter desempenho mais estável em instâncias maiores.

6.2.2 Distância das rotas

Os resultados são analisados em termos de distância total da rota, considerando a proximidade com a solução ótima e destacando as diferenças entre as abordagens exatas, heurísticas e quânticas. Essa análise permite compreender o comportamento de cada método diante de diferentes cenários de complexidade, servindo como base para a comparação detalhada apresentada nas seções seguintes.

Abaixo, na Tabela 14, serão disponibilizados os resultados, e na sequência, apresentados como gráfico na Figura 21.

Figura 21 - Gráfico de distância das rotas dos algoritmos



Fonte: autoria propria

Tabela 14 - Distância das rotas dos algoritmos

Pontos	FB (s)	AG (s)	SA (s)	IQAOA clássico (s)	IQAOA quântico (s)
5	6940,59	6940,59	6940,59	6940,59	6940,59
10	10165,18	10165,18	10165,18	13333,59	13333,59
20	–	15683.62	14904.97	25945.14	26140.1

Fonte: autoria própria.

Com base nos resultados apresentados na Tabela 14, observa-se que, para a instância de 5 pontos, todos os algoritmos analisados força bruta, algoritmo genético, *simulated annealing*, IQAOA clássico e quântico convergiram para a mesma distância total de rota (6940,59 m). Isso indica que, em instâncias pequenas, mesmo métodos heurísticos ou quânticos são capazes de encontrar soluções ótimas ou muito próximas do ótimo, mostrando que a complexidade do problema ainda é suficientemente baixa para que diferentes abordagens alcancem resultados equivalentes.

No caso da instância de 10 pontos, os algoritmos clássicos (BF, GA e SA) continuaram a convergir para o mesmo valor de 10165,18 m, enquanto o IQAOA (clássico e quântico) apresentou uma distância maior (13333,59 m). Isso sugere que, para esse problema específico, o IQAOA ainda não atingiu a solução ótima,

possivelmente em função do aumento do tamanho da instância ou das configurações dos hiper parâmetros do algoritmo. Ainda assim, observou-se consistência entre as versões clássica e quântica do IQAOA.

Esses resultados destacam que, em instâncias pequenas, há boa convergência entre diferentes abordagens, permitindo uma comparação direta entre algoritmos exatos, heurísticos e quânticos.

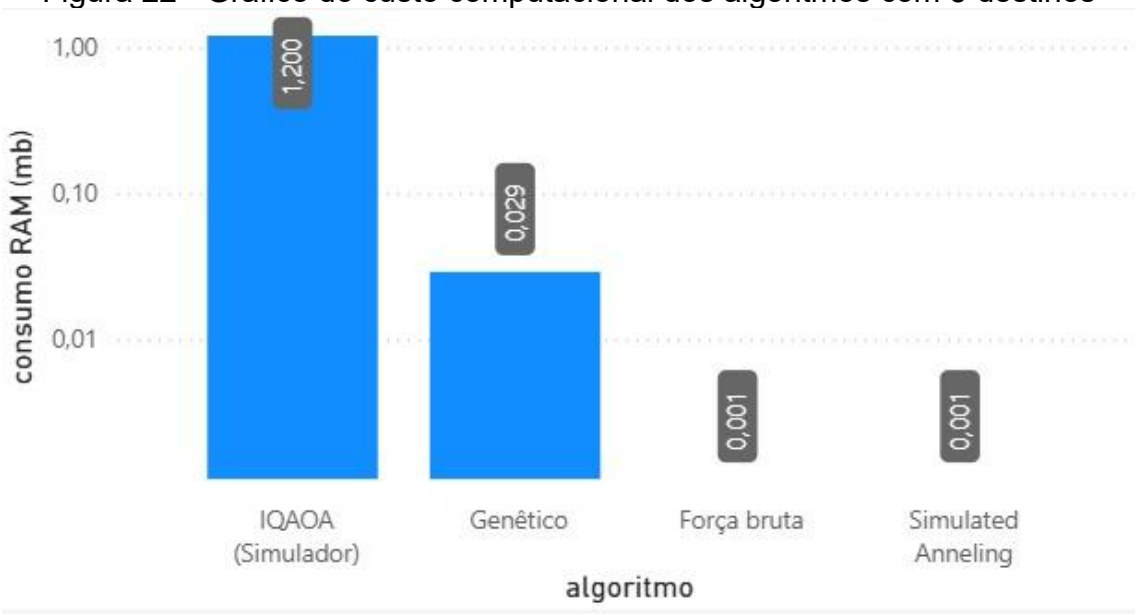
6.2.3 Custo computacional

Neste subcapítulo será apresentada a análise do custo computacional dos algoritmos aplicados ao PCV, considerando diferentes tamanhos de instâncias. Serão avaliados os recursos necessários para execução, incluindo processamento clássico e quântico, número de qubits, simulação versus *hardware* real, e as restrições práticas de cada abordagem.

6.2.3.1 Custo computacional para 5 pontos

Avaliação de desempenho dos algoritmos para um conjunto de 5 pontos de destino, permitindo observar o comportamento do consumo de recursos em problemas mais simples, como apresentado na Figura 22.

Figura 22 - Gráfico do custo computacional dos algoritmos com 5 destinos



Fonte: autoria própria.

Tabela 15 - Custo computacional dos algoritmos com 5 destinos

Algoritmo	Consumo *RAM (mb)*	Nº de qubits
Força bruta	0,0011	–
Algoritmo genético	0,0289	–
IQAOA (clássico)	1,21	5
IQAOA (quântico)	–	5
Simulated annealing	0,0011	–

Fonte: autoria própria.

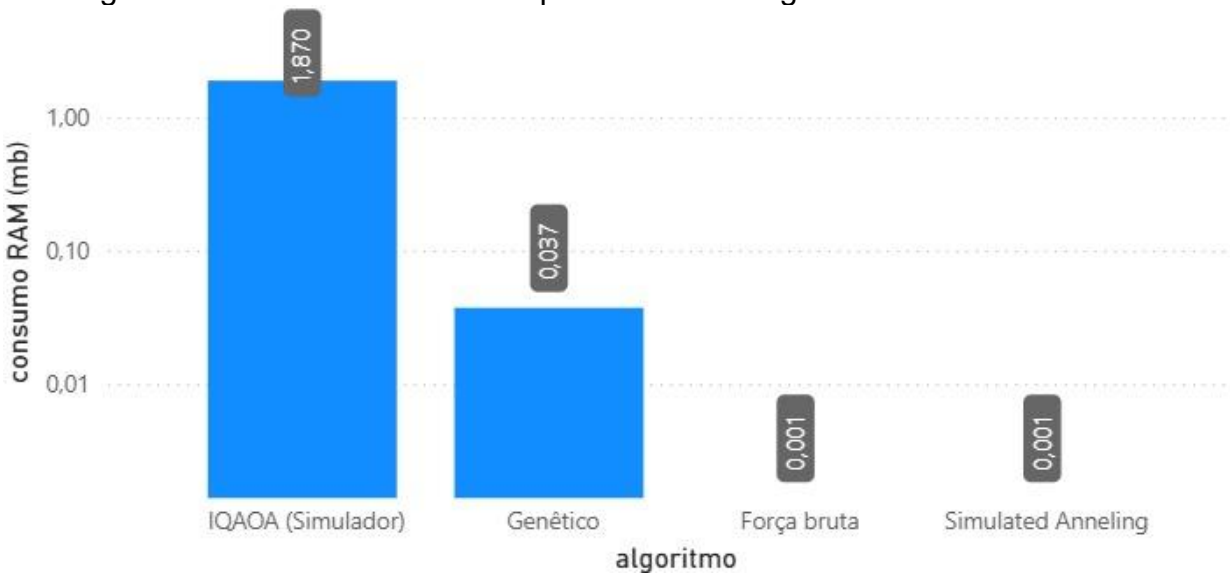
Observa-se que os algoritmos clássicos apresentam baixo consumo de memória, com destaque para o força bruta e o *simulated annealing*, ambos utilizando cerca de 0,0011 MB, evidenciando sua leveza computacional. O algoritmo genético demanda um pouco mais, 0,0289 MB, reflexo do armazenamento das populações durante o processo evolutivo. Já o IQAOA em simulador apresenta consumo significativamente superior, 1,21 MB, devido à necessidade de representar e manipular estados quânticos em ambiente clássico. No caso do IQAOA executado em *hardware* quântico, o consumo de RAM local é desprezível, uma vez que o processamento ocorre diretamente no dispositivo quântico.

Sobre os qubits, tanto o simulador quanto o *hardware* quântico irão executar o mesmo circuito, logo o número de qubits não mudam.

6.2.3.2 Custo computacional para 10 pontos

Avaliação do desempenho dos algoritmos para um conjunto de 10 pontos de destino, evidenciando o impacto do aumento do número de pontos sobre o consumo de processamento e memória, bem como a necessidade de qubits no caso do IQAOA, conforme ilustrado na Figura 23.

Figura 23 - Gráfico do custo computacional dos algoritmos com 10 destinos



Fonte: autoria própria.

Tabela 16 - Custo computacional dos algoritmos com 10 destinos

Algoritmo	**Consumo *RAM (mb)***	**Nº de *qubits***
Força bruta	0,0014	–
Algoritmo genético	0,0372	–
IQAOA (clássico)	1,87	10
IQAOA (quântico)	–	10
Simulated annealing	0,0014	–

Fonte: autoria própria.

No segundo cenário, nota-se novamente que os algoritmos clássicos mantêm baixo consumo de memória em geral. O força bruta e o *simulated annealing* continuam empatados em menor consumo de memória. O algoritmo genético apresenta um consumo ligeiramente maior, 0,0372 MB, coerente com o aumento da complexidade populacional à medida que o problema cresce. Já o IQAOA em simulador demonstra um salto expressivo no uso de RAM, alcançando 1,81 MB, consequência direta do crescimento do número de qubits e da complexidade da simulação quântica em ambiente clássico.

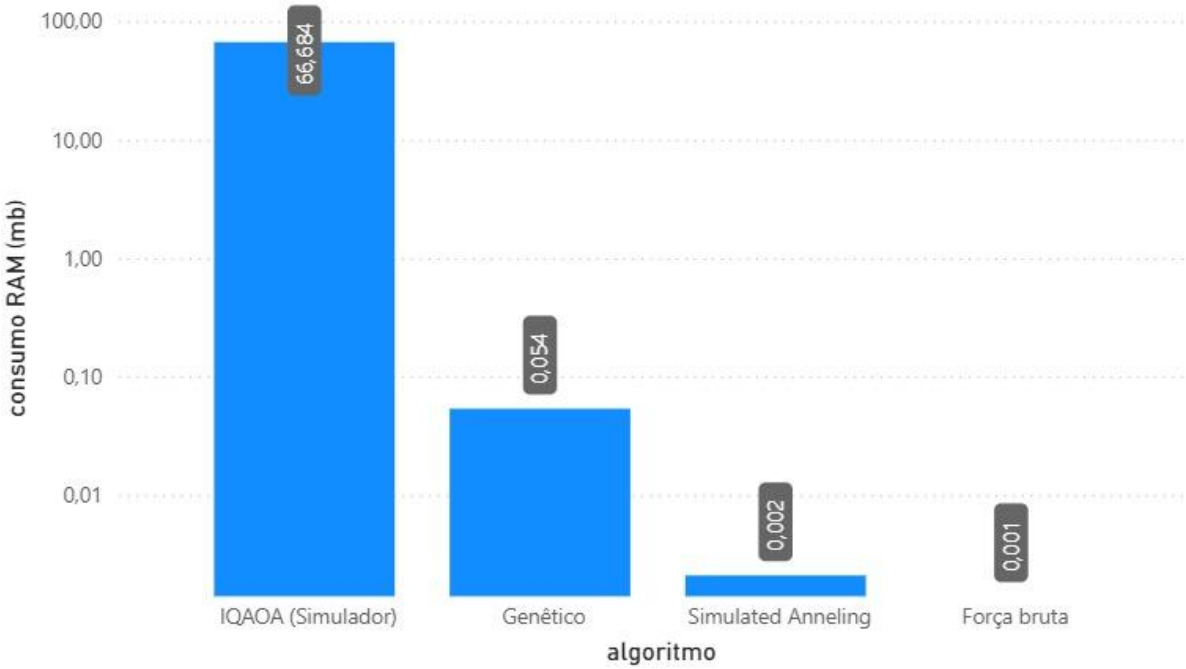
No IQAOA executado em *hardware* quântico, o consumo de RAM local permanece irrelevante, pois o processamento é realizado no próprio dispositivo quântico

Como já explicado anteriormente, o simulador e o *hardware* quântico, se assemelham em quantidade de qubits.

6.2.3.2 Custo computacional para 20 pontos

Avaliação do desempenho dos algoritmos para um conjunto de 20 pontos de destino, destacando como o aumento da complexidade influencia o consumo de processamento, memória e a quantidade de qubits necessários para o IQAOA, conforme ilustrado na Figura 24.

Figura 24 - Gráfico do custo computacional dos algoritmos com 20 pontos



Fonte: autoria própria.

Tabela 17 - Custo computacional dos algoritmos com 20 pontos

Algoritmo	**Consumo *RAM (mb)***	**Nº de *qubits***
Força Bruta	–	–
Algoritmo Genético	0.0537	–
IQAOA (clássico)	66.6841	20
IQAOA (quântico)	–	20
Simulated Annealing	0.0021	–

Fonte: autoria própria.

No terceiro cenário, com 20 pontos, observa-se que os algoritmos clássicos mantêm baixo consumo de memória em geral. O *simulated annealing* continua apresentando o menor consumo, 0,0021 MB, enquanto o Algoritmo Genético registra um valor ligeiramente maior, 0,0537 MB, refletindo o aumento da complexidade populacional frente ao crescimento do problema.

Já o IQAOA em simulador clássico apresenta um salto expressivo no uso de RAM, alcançando 66,6841 MB, devido à necessidade de simular 20 qubits e à complexidade associada à computação quântica em ambiente clássico. No IQAOA executado em *hardware* quântico, o consumo de memória local não é relevante, pois o processamento ocorre diretamente no dispositivo quântico. Assim, apesar do aumento do número de qubits, tanto o simulador quanto o *hardware* quântico mantêm a quantidade de qubits, evidenciando a diferença entre a sobrecarga de memória em simulação clássica e execução real em *hardware* quântico.

6.2.4 Custo Monetário

Neste subcapítulo, será analisado o custo monetário associado à execução dos algoritmos, considerando o impacto financeiro das diferentes abordagens computacionais, tanto clássicas quanto quânticas.

Para definição da métrica de preço, adotou-se como referência o modelo de cobrança da *AWS Braket* para execuções em QPU, que estabelece um custo fixo de US$ 0,30 por tarefa quântica submetida e US$ 0,0009 por *shot* no computador Ankaa-3. Para execuções em CPU, considerou-se o modelo de cobrança da AWS baseado no tempo de processamento por minuto, com valor de US$ 0,075.

Além disso, tanto o IQAOA quântico quanto o IQAOA em simulador seguiram a métrica de custo da QPU, visto que o simulador busca reproduzir o comportamento de uma execução quântica real, garantindo coerência na comparação entre as abordagens. Embora a etapa de otimização de parâmetros do algoritmo quântico tenha sido realizada no simulador clássico para reduzir o custo do experimento, todos os cálculos foram orçados como se a otimização tivesse ocorrido totalmente em ambiente quântico.

Essa etapa é a mais custosa porque o otimizador COBYLA, ao ajustar os parâmetros, utiliza o valor de *maximiter* para gerar múltiplas tarefas, cada uma contendo um número definido de *shots*, estimando o valor da função de custo em cada

iteração. Como resultado, a quantidade total de tarefas submetidas ao *back-end* quântico cresce rapidamente, impactando significativamente o custo de execução.

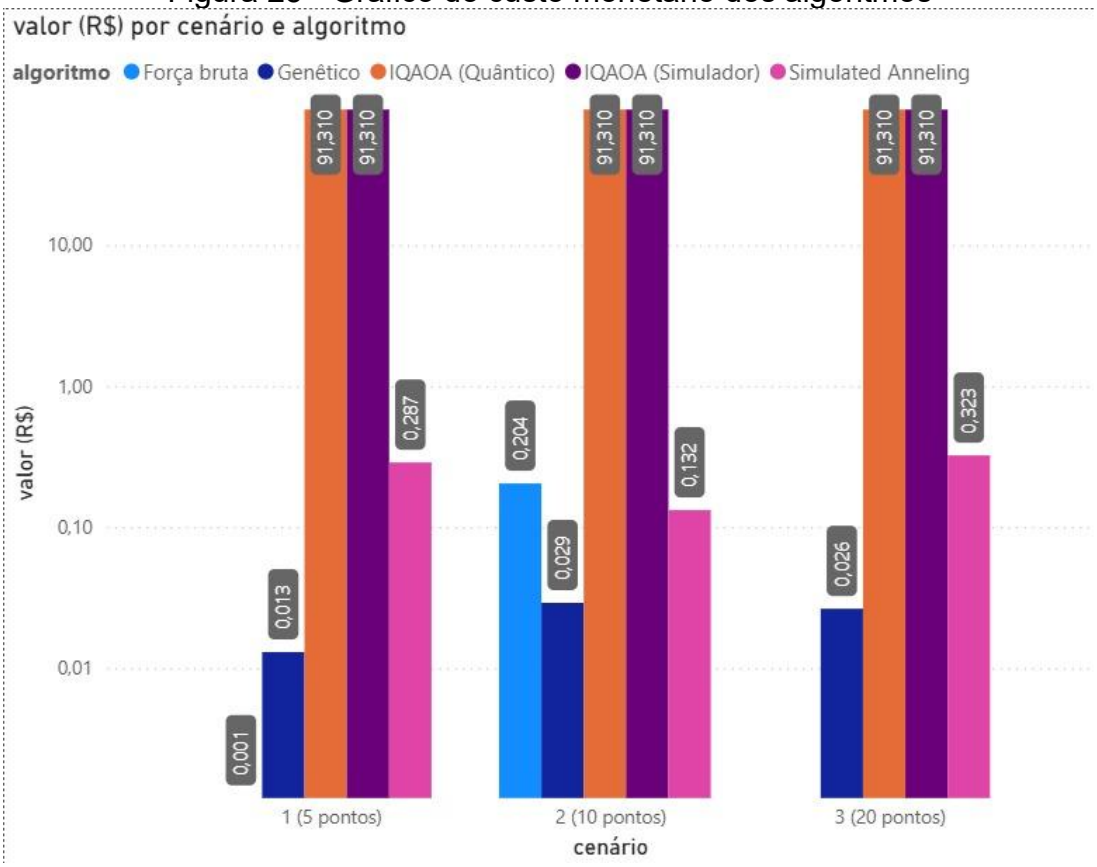
Para conversão de dólar para real, utilizou-se a cotação de 1 USD = 5,34 BRL. A seguir, o custo estimado para cada algoritmo é apresentado na Tabela 18 e ilustrado na Figura 25 permitindo uma análise comparativa clara entre as diferentes abordagens.

Tabela 18 - Custo monetário dos algoritmos

Algoritmo	**5 pontos**	**10 pontos**	**20 pontos**
Força Bruta	0,000002	0,204	–
Algoritmo Genético	0,013	0,02904	0.026387327
IQAOA (clássico)	91,31	91,31	91.314
IQAOA (quântico)	91,31	91,31	91.314
Simulated Annealing	0,132	0,2873	0.322502493

Fonte: autoria própria.

Figura 25 - Gráfico do custo monetário dos algoritmos



Fonte: autoria própria

Como observado na Figura 25, há uma diferença marcante entre os algoritmos clássicos e quânticos. O algoritmo Força Bruta apresenta crescimento exponencial no custo, passando de valores desprezíveis para significativamente elevados à medida que o número de pontos aumenta, refletindo a limitada escalabilidade desse método. O algoritmo genético e o *simulated annealing*, por sua vez, mantêm custos baixos e proporcionais ao aumento da complexidade, evidenciando sua eficiência em termos de custo-benefício.

Em contraste, tanto o IQAOA clássico quanto o quântico apresentam custos relativamente fixos, independentemente do tamanho da instância. Isso ocorre porque a métrica de preço considera os parâmetros do algoritmo, que impactam diretamente na quantidade de tarefas e shots submetidos. Como ambos os algoritmos utilizaram os mesmos parâmetros, os custos foram equivalentes.

Esses resultados ressaltam a inviabilidade do uso de tecnologia quântica em problemas de baixa complexidade, nos quais o número de combinações pode ser resolvido de forma eficiente por algoritmos clássicos, sem a necessidade do elevado custo computacional e financeiro associado às abordagens quânticas.

6.3 DISCUSSÃO GERAL DOS RESULTADOS

A análise dos resultados obtidos permite observar padrões claros no desempenho e na aplicabilidade de cada algoritmo frente ao PCV. A avaliação comparativa, abrangendo métricas de tempo de execução, qualidade da rota (distância), custo computacional (memória RAM) e custo monetário.

As subseções a seguir aprofundam a discussão, primeiro comparando o desempenho entre os diferentes algoritmos e, em seguida, analisando as nuances da execução do IQAOA em ambientes clássico (simulador) e quântico (*hardware* real).

6.3.1 Entre os algoritmos

A comparação direta entre as abordagens de força bruta, algoritmo genético, *simulated annealing* e IQAOA evidencia distinções claras de desempenho, que se acentuam com o aumento da complexidade do problema. Para o cenário de 5 pontos, uma instância de baixa complexidade, todos os algoritmos, incluindo os heurísticos e

o quântico, foram capazes de convergir para a solução ótima, com uma distância de 6940,59 m. Isso demonstra que, para problemas com um espaço de busca reduzido, a complexidade é suficientemente baixa para que diferentes estratégias alcancem o mesmo resultado de alta qualidade.

No cenário de 10 pontos, as diferenças se tornaram evidentes. Enquanto os algoritmos clássicos (força bruta, algoritmo genético e *simulated annealing*) ainda encontraram a rota ótima de 10165,18 m, o IQAOA, tanto no simulador quanto no *hardware* quântico, convergiu para uma solução ótima de 13333,59 m. Este resultado sugere que, com os parâmetros configurados, o IQAOA não conseguiu explorar o espaço de soluções de forma tão eficaz quanto as heurísticas clássicas para esta instância. Em termos de tempo de execução, a força bruta e o *simulated annealing* apresentaram aumento significativo, enquanto o algoritmo genético e o IQAOA mantiveram escalabilidade temporal mais favorável.

No terceiro cenário, com 20 pontos, o problema tornou-se ainda mais desafiador. A força bruta se tornou inviável, devido ao crescimento exponencial do espaço de busca, estimando-se cerca de $1{,}21645 \times 10^{17}$ permutações, o que resultaria em aproximadamente 1.680 anos de execução contínua em um processador Intel Core i7 de 2,30 GHz. Diante disso, apenas os algoritmos genético, *simulated annealing* e IQAOA puderam ser executados. A análise revelou que o *simulated annealing* obteve a melhor rota em termos de distância total (14.904,97 m), seguido pelo algoritmo genético (15.683,62 m, aproximadamente 5,2% maior) e pelo IQAOA, que apresentou a solução menos competitiva. Em termos de tempo de execução, o algoritmo genético foi o mais rápido (3,95 s), enquanto o *simulated annealing* demandou 48,31 s, e o IQAOA manteve um tempo de execução intermediário.

A análise de consumo de memória e custo monetário, conforme a Tabela 17, evidencia que os algoritmos clássicos mantêm baixo consumo de RAM, sendo o *simulated annealing* o mais eficiente, seguido pelo algoritmo genético. O IQAOA em simulação exigiu grande quantidade de memória (66,6841 MB) devido ao aumento do número de qubits, enquanto no *hardware* quântico, o consumo de RAM local é irrelevante, pois o processamento é realizado no dispositivo quântico. O custo monetário da abordagem quântica permanece elevado e fixo, independentemente do tamanho do problema, devido à quantidade de tarefas e shots necessários.

Esses resultados reforçam que, para instâncias de pequena e média escala, os algoritmos clássicos são mais eficientes em termos de tempo, memória e custo,

enquanto as abordagens quânticas ainda enfrentam limitações significativas. Contudo, para problemas de grande porte, como o cenário de 20 pontos, os heurísticos como *simulated annealing* e algoritmo genético permanecem viáveis, enquanto a aplicação de IQAOA em *hardware* quântico permanece restrita pela disponibilidade de qubits e capacidade do processamento quântico atual.

6.3.2 Entre ambientes de execução

A análise do IQAOA em dois ambientes distintos simulador clássico e *hardware* quântico oferece importantes *insights* sobre o estado atual da computação quântica. O simulador clássico se destaca por fornecer resultados consistentes e confiáveis, sendo uma ferramenta essencial para o desenvolvimento e depuração de algoritmos quânticos.

Por outro lado, a execução em *hardware* quântico enfrenta desafios inerentes à era NISQ, como ruído e decoerência, que podem introduzir variabilidade e reduzir a precisão dos resultados. Apesar dessas limitações, o IQAOA em *hardware* quântico foi capaz de gerar a solução ótima para a instância de 5 pontos, demonstrando seu potencial para produzir resultados competitivos. A consistência observada entre os resultados do simulador e do *hardware* quântico reforça a validade do circuito quântico projetado.

A abordagem híbrida adotada, na qual a otimização dos parâmetros é realizada no simulador e a amostragem final ocorre no dispositivo quântico, mostrou-se uma estratégia prática. Essa metodologia permite mitigar os altos custos associados à execução iterativa em uma QPU, etapa mais custosa do processo, mantendo a eficiência sem comprometer a qualidade da solução.

7 CONSIDERAÇÕES FINAIS

Por meio da análise comparativa dos resultados apresentados no capítulo de validação, especialmente nos experimentos com 20 pontos, observa-se que a diferença de desempenho entre os ambientes clássico e quântico tende a diminuir conforme a complexidade do problema aumenta. No cenário com 5 e 10 pontos, o simulador clássico apresentou tempos de execução de 1,6055 s e 2,6692 s, respectivamente, sendo aproximadamente três vezes mais rápido que o *hardware* quântico, que obteve 5,739 s e 6,122 s. Contudo, ao atingir 20 pontos, o tempo de execução do IQAOA clássico (644,1378 s) superou consideravelmente o quântico (425,69 s), indicando uma possível inversão de desempenho. Esse comportamento reforça que, embora os ambientes clássicos ainda sejam mais estáveis e adequados para análises combinatórias de pequena e média escala, os ambientes quânticos demonstram um potencial crescente para lidar com problemas de maior complexidade e volume de dados.

Diante disso, conclui-se que o modelo de processamento mais adequado depende diretamente da natureza e da dimensão do problema a ser resolvido. Para instâncias menores e de rápida convergência, os métodos clássicos, como força bruta, genético e *simulated annealing*, continuam sendo as opções mais eficientes e acessíveis. Já em cenários com um número elevado de variáveis, em que o espaço de busca cresce exponencialmente, os algoritmos quânticos, como o IQAOA, tendem a se destacar, principalmente à medida que o avanço tecnológico reduzir as limitações de ruído e decoerência dos qubits. Assim, a computação quântica, embora ainda em desenvolvimento, mostra-se promissora para o futuro da otimização combinatória, podendo superar as abordagens clássicas à medida que sua maturidade e escalabilidade forem aprimoradas.

REFERÊNCIAS

ASPURU-GUZIK, Alán et al. **Simulated quantum computation of molecular energies.** ***Science***, v. 309, n. 5741, p. 1704–1707, 2005. Disponível em: https://arxiv.org/abs/quant-ph/0604193. Acesso em: 31 ago. 2025.

BERNHARDT, Chris. **Quantum computing for everyone.** 1. ed. Cambridge: MIT Press, 2020.

BHARTI, Kishor et al. **Noisy intermediate-scale quantum algorithms**. *Reviews of Modern Physics*, v. 94, n. 1, p. 015004, 2022. Disponível em: https://journals.aps.org/rmp/abstract/10.1103/RevModPhys.94.015004. Acesso em: 31 ago. 2025.

BIAMONTE, Jacob et al. **Quantum machine learning.** ***Nature***, v. 549, n. 7671, p. 195–202, 2017. Disponível em: https://www.nature.com/articles/nature23474. Acesso em: 31 ago. 2025.

BOURREAU, Eric; FLEURY, Gérard; LACOMME, Philippe. **IQAOA for Two Routing Problems: A Methodological Contribution with Application to TSP and VRP.** Journal of Quantum Computing, v. 6, p. 25, 2024.

BRASSARD, Gilles et al. **Quantum computing**. *Proceedings of the National Academy of Sciences*, v. 95, n. 19, p. 11032–11033, 1998. Disponível em: https://www.pnas.org/doi/abs/10.1073/pnas.95.19.11032. Acesso em: 31 ago. 2025.

CATARINO, Marino. **Teoria dos grafos.** 1. ed. São Paulo: Blucher, 2023.

CHEN, Lily et al. **Report on post-quantum cryptography. Gaithersburg, MD, USA: US Department of Commerce, National Institute of Standards and Technology**, 2016. Disponível em: https://nvlpubs.nist.gov/nistpubs/ir/2016/NIST.IR.8105.pdf?ref=suddo.de. Acesso em: 31 ago. 2025.

EGGERT, Jannik. **Solving the Traveling Salesman Problem with the Quantum Approximate Optimization Algorithm.** 2023.

FERNÁNDEZ-PENDÁS, Mario et al. **A study of the performance of classical minimizers in the quantum approximate optimization algorithm.** Journal of Computational and Applied Mathematics, v. 404, p. 113388, 2022.

GOTTESMAN (ProQuest – ProQuest LLC). **Stabilizer codes and quantum error correction,** California, EUA, 1997. Disponível em: https://www.proquest.com/openview/82213d22fc3a65405da6fe9a2106c432/1?pq-origsite=gscholar&cbl=18750&diss=y. Acesso em: 31 ago. 2025.

GROVER, Lov K. **Synthesis of quantum superpositions by quantum computation.** ***Physical Review Letters***, v. 85, n. 6, p. 1334, 2000. Disponível em: https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.85.1334. Acesso em: 31 ago. 2025.

HORODECKI, Ryszard et al. **Quantum entanglement.** ***Reviews of Modern Physics***, v. 81, n. 2, p. 865–942, 2009. Disponível em: https://journals.aps.org/rmp/abstract/10.1103/RevModPhys.81.865. Acesso em: 31 ago. 2025.

Marcos Paulo Ferreira de Araújo. **Introdução à Teoria dos Grafos - Aula 1 - O que é um grafo?.** [S.l.:s.n.], 2016. 1 vídeo (11min 30seg). Publicado pelo canal Programa de Iniciação Cientifica da OBMEP Português. Disponível em: https://youtu.be/Frmwdter-vQ?si=8ITuXesjD_osVGsV . Acesso em: 22 mar. 2025.

NASCIMENTO, João Paulo B.; MURTA, Cristina D. **Um algoritmo paralelo em Hadoop para cálculo de centralidade em grafos grandes**. In: SIMPÓSIO BRASILEIRO DE REDES DE COMPUTADORES E SISTEMAS DISTRIBUÍDOS, 30., 2025. Anais [...]. Belo Horizonte, MG: CEFET-MG, 2025. Disponível em: https://ce-resd.sbc.org.br/sbrc/2012/ST8_3.pdf. Acesso em: 04 abr. 2025.

NIELSEN, Michael A.; CHUANG, Isaac L. **Quantum computation and quantum information, Cambridge, Cambridge University Press**, 2010. Disponível em: https://books.google.com.br/books?id=-s4DEy7o-a0C&lpg=PR17&ots=NJ6KipmBTr&dq=NIELSEN%2C%20Michael%20A.%3B%20CHUANG%2C%20Isaac%20L.%20Quantum%20computation%20and%20quantum%20information.%20Cambridge%20university%20press%2C%202010.&lr&hl=pt-BR&pg=PR6#v=onepage&q&f=false. Acesso em: 31 ago. 2025.

OLIVEIRA, Marciel Medeiros de. **Aplicações do teorema de Shemesh e dos Conceitos de subespaço invariante e canal quântico não-ergódico na teoria da informação quântica erro-zero.** 2024. 101 f. Tese (Doutorado em Engenharia Elétrica) – Programa de Pós-Graduação em Engenharia Elétrica, Centro de Engenharia Elétrica e Informática, Universidade Federal de Campina Grande, Campina Grande, 2024. Disponível em: http://dspace.sti.ufcg.edu.br:8080/jspui/handle/riufcg/40445 . Acesso em: 29 mar. 2025.

PERKINS, R. **Travelling salesman problem: the optimization challenge to rule them all.** Medium, [S. l.], 2021. Disponível em: https://randyperkins2k.medium.com/travelling-salesman-problem-the-optimization-challenge-to-rule-them-all-1f69738f598b. Acesso em: 20 abr. 2025.

PRESKILL, John. **Quantum computing in the NISQ era and beyond**. *Quantum*, v. 2, p. 79, 2018. Disponível em: https://quantum-journal.org/papers/q-2018-08-06-79/pdf/. Acesso em: 31 ago. 2025.

PRESKILL, John. **Reliable quantum computers.** ***Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences***, v. 454, n. 1969, p. 385–410, 1998. Disponível em: https://royalsocietypublishing.org/doi/abs/10.1098/rspa.1998.0167. Acesso em: 31 ago. 2025.

Luiz Phellype Alves. **Realidade em Perigo: A Ameaça Silenciosa dos Computadores Quânticos.** [S.l.:s.n.], 2025. 1 vídeo (8min 17seg). Publicado pelo canal Fatos Desconhecidos Português. Disponível em: https://youtu.be/vLbbTpfzzFc?si=BguLsIZxG_AsPmke . Acesso em: 22 mar. 2025.

REVISTA BRASILEIRA DE ENSINO DE FÍSICA. **Uma Introdução à Ciência de Redes e Teoria de Grafos**. Revista Brasileira de Ensino de Física, São Paulo, v. 42, n. 1, p. 1-10, jan./mar. 2020. Disponível em: https://www.scielo.br/j/rbef/a/GLGWtVsYZ7zysCBc3Xktf6b/?lang=pt. Acesso em: 23 mar. 2025.

RODRIGUES, William Wallace Teodoro; FABRÍCIO FILHO, João. **Avaliação de Desempenho Temporal em Computação Quântica: Estudo sobre Transpilação e Execução em Circuitos de Superposição e Entrelaçamento**. In: ESCOLA REGIONAL DE ALTO DESEMPENHO DE SÃO PAULO (ERAD-SP), 16., 2025, São José do Rio Preto/SP. Anais [...]. Porto Alegre: Sociedade Brasileira de Computação, 2025. p. 46-49. Disponível em: https://sol.sbc.org.br/index.php/eradsp/article/view/36415.

SANTOS, Marcelo de Souza. **Ciclos Hamiltonianos em grafos**. 2016. Monografia (Trabalho de Conclusão de Curso em Matemática) – Instituto de Matemática, Universidade Federal do Rio Grande do Sul, Porto Alegre, 2016. Disponível em: https://lume.ufrgs.br/handle/10183/150239. Acesso em: 20 abr. 2025.

SANTOS, Thiago A. dos; PEREIRA, Marcelo A.; MOREIRA, Marco A. **Computação quântica: uma abordagem para a graduação usando o *Qiskit***. Revista Brasileira de Ensino de Física, São Paulo, v. 44, e20220126, 2022. Disponível em: https://www.scielo.br/j/rbef/a/qRcz9vCVfYMv4J4rDjKqLtP/?lang=pt. Acesso em: 16 mar. 2025.

SCHMIDT, Dionatan Ricardo. **Problemas de coloração em grafos evitando famílias de padrões de grafos completos**. 2023. Tese (Doutorado em Matemática Aplicada) – Programa de Pós-Graduação em Matemática Aplicada, Universidade Federal do Rio Grande do Sul, Porto Alegre, 2023. Disponível em: http://hdl.handle.net/10183/259369. Acesso em: 29 mar. 2025

SILVA, M. A. da; OLIVEIRA, R. A. de; COSTA, J. P. da**. Grafos: conceitos fundamentais, algoritmos e aplicações.** 1. ed. Santa Catarina: Editora IFC, 2022. Disponível em: https://editora.ifc.edu.br/wp-content/uploads/sites/39/2022/11/Grafos-versao-final.pdf. Acesso em: 20 abr. 2025.

STORINO, Lucas Bezerra; MARQUEZINO, Franklin. **Computação quântica e aprendizado de máquina: uma abordagem quântica em grafos**. In: JORNADA MASSARANI DE INICIAÇÃO CIENTÍFICA, TECNOLÓGICA, ARTÍSTICA E CULTURAL, 22., 2021, Rio de Janeiro. Anais [...] Rio de Janeiro: UFRJ, 2021. p. 1-10. Disponível em: https://www.even3.com.br/anais/jgmictac/317084-computacao-quantica-e-aprendizado-de-maquina--uma-abordagem-quantica-em-grafos. Acesso em: 29 mar. 2025.

VAKILI, Mohammad Ghazi et al. **Quantum computing-enhanced algorithm unveils novel inhibitors for KRAS**. *arXiv preprint arXiv:2402.08210*, 2024. Disponível em: https://arxiv.org/abs/2402.08210. Acesso em: 31 ago. 2025.

ZHOU, A.-H.; ZHU, L.-P.; HU, B.; DENG, S.; SONG, Y.; QIU, H.; PAN, S. **Traveling-salesman-problem algorithm based on simulated annealing and gene-expression programming**. Information, [S. l.], v. 10, n. 1, p. 7, 2019. Disponível em: https://doi.org/10.3390/info10010007. Acesso em: 20 abr. 2025.